



UNIVERSIDAD
PERUANA LOS ANDES

Facultad de Ingeniería

ADMINISTRACIÓN, OPTIMIZACIÓN Y MONITOREO DE BASES DE DATOS EN SQL SERVER



Proyección aplicada sobre
la base de datos **GradosTitulos**

TEMAS:

-  Estadísticas y Eventos
-  Estadísticas e Índices
-  Transacciones y Bloqueos
-  Planes de Ejecución
-  Optimización T-SQL
-  Resource Governor



MONITOREA
y diagnostica
el rendimiento



OPTIMIZA
consultas
e índices



CONTROLA
transacciones
y bloqueos



ANALIZA
planes de
ejecución



ADMINISTRA
recursos de forma
eficiente

DATOS FUERTES, DECISIONES INVENCIBLES.



TEMA 1. DIAGNÓSTICO Y SEGUIMIENTO DEL RENDIMIENTO EN SQL SERVER

Proyecto 1: Seguimiento del rendimiento mediante Extended Events

1. Enunciado

El área de Tecnologías de la Información de la universidad necesita detectar las consultas que demoran demasiado al ejecutarse sobre la base GradosTitulos. Para lograrlo, se debe configurar una sesión de Extended Events que capture toda consulta cuya duración supere los 1000 ms, almacenando los registros en un archivo .xel que luego permita elaborar un reporte con datos como fecha, usuario responsable, duración, consumo de CPU, lecturas lógicas y el texto de la consulta ejecutada.

2. Script de referencia

```

DROP EVENT SESSION XE_P1_ConsultasCostosas ON SERVER;
GO

DECLARE @LogPath NVARCHAR(500);
DECLARE @SQL NVARCHAR(MAX);

SELECT @LogPath =
LEFT(errorlog,
LEN(errorlog)-CHARINDEX('\', REVERSE(errorlog))+1)
FROM
(
SELECT CAST(SERVERPROPERTY('ErrorLogFileName') AS NVARCHAR(500)) AS errorlog
)x;

SET @SQL='

CREATE EVENT SESSION XE_P1_ConsultasCostosas
ON SERVER

ADD EVENT sqlserver.sql_statement_completed
(
ACTION
(
sqlserver.sql_text,
sqlserver.database_name,
sqlserver.session_id,
sqlserver.username,
sqlserver.client_hostname
)

WHERE
(
(sqlserver.database_name='GradosTitulos')
AND duration > 1000000
)
)

ADD TARGET package0.event_file
(
SET filename=''+@LogPath+'XE_P1_ConsultasCostosas.xel'
);

```

```

SET filename=''+@LogPath+'XE_P1_ConsultasCostosas.xel'
);

EXEC(@SQL);
GO

ALTER EVENT SESSION XE_P1_ConsultasCostosas
ON SERVER
STATE=START;
GO

SELECT p.IdPersona,
       p.Nombres,
       p.ApellidoPaterno,
       m.IdMatricula,
       t.IdTramite
FROM Academico.Persona p
INNER JOIN Academico.Matricula m
ON p.IdPersona=m.IdEstudiante
INNER JOIN Tramite.Tramite t
ON m.IdMatricula=t.IdMatricula;

WAITFOR DELAY '00:00:05';
GO

ALTER EVENT SESSION XE_P1_ConsultasCostosas
ON SERVER
STATE=STOP;
GO

```

Imagen 1

Ubicación de la sesión

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Explorador de objetos' (Object Explorer) pane displays the server hierarchy for 'EITR\EITR (17.0.1115.1 de SQL Server - EITR\Jefte)'. The 'Eventos extendidos' (Extended Events) folder is expanded, showing the 'XE_P1_ConsultasCostosas' session. The 'package0.event_file' file is highlighted under this session.

On the right, the 'SQLQuery3.sql...R\Jefte (71)' window displays the SQL script from the previous image. Below the script, the 'Resultados' (Results) pane shows the output of the query, which is a table with 6 columns: 'IdPersona', 'Nombres', 'ApellidoPaterno', 'IdMatricula', and 'IdTramite'. The table contains 13 rows of data, numbered 564 to 577.

At the bottom, a status bar indicates 'Consulta ejecutada correctamente.' (Query executed successfully).

	IdPersona	Nombres	ApellidoPaterno	IdMatricula	IdTramite
564	5010	Andrea	Ramos	4860	564
565	5031	Luis	Rivera	4881	565
566	5048	Milagros	Gutierrez	4898	566
567	5065	Raul	Mendoza	4915	567
568	5074	Patricia	Lopez	4924	568
569	5085	Luis	Gonzalez	4935	569
570	5099	Diego	Torres	4949	570
571	5102	Milagros	Gomez	4952	571
572	5111	Diego	Castillo	4961	572
573	5118	Andrea	Ramirez	4968	573
574	5121	Luis	Rivera	4971	574
575	5129	Diego	Torres	4979	575
576	5132	Milagros	Gomez	4982	576
577	5143	Raul	Diaz	4993	577

Justificación técnica :

Se configuró una sesión de **Extended Events** con el propósito de supervisar las consultas que excedieran los **1000 ms** de tiempo de ejecución en la base de datos **GradosTitulos**. Los eventos registrados se guardaron en un archivo con extensión **.xel**, lo que facilitó el análisis posterior de las consultas con mayor impacto en el rendimiento, minimizando al mismo tiempo el consumo de recursos del servidor.

Buenas practicas :

- Se asignó a la sesión un nombre representativo (**XE_P1_ConsultasCostosas**) para facilitar su identificación y administración.
- Se configuró un filtro para registrar únicamente los eventos correspondientes a la base de datos **GradosTitulos**, excluyendo información que no fuera relevante para el análisis.
- La ubicación donde se almacenó el archivo **.xel** se obtuvo de forma automática, evitando inconvenientes relacionados con rutas inválidas o restricciones de acceso.
- La sesión permaneció activa únicamente durante el periodo de ejecución de las pruebas y posteriormente fue finalizada, con el fin de optimizar el uso de los recursos del servidor.

Proyecto 2: Implementación de Backups Diferenciales para Trámites Académicos

1. Enunciado

Durante los procesos de matrícula para obtención del título profesional, varios usuarios reportan tiempos elevados en la generación de resoluciones.

La Dirección de Informática solicita construir una solución de monitoreo que capture automáticamente:

- Consultas lentas.
- Deadlocks
- Bloqueos.
- Errores superiores al nivel 16.
- Uso excesivo de CPU

. Toda la información deberá almacenarse en un archivo de eventos para su posterior análisis.

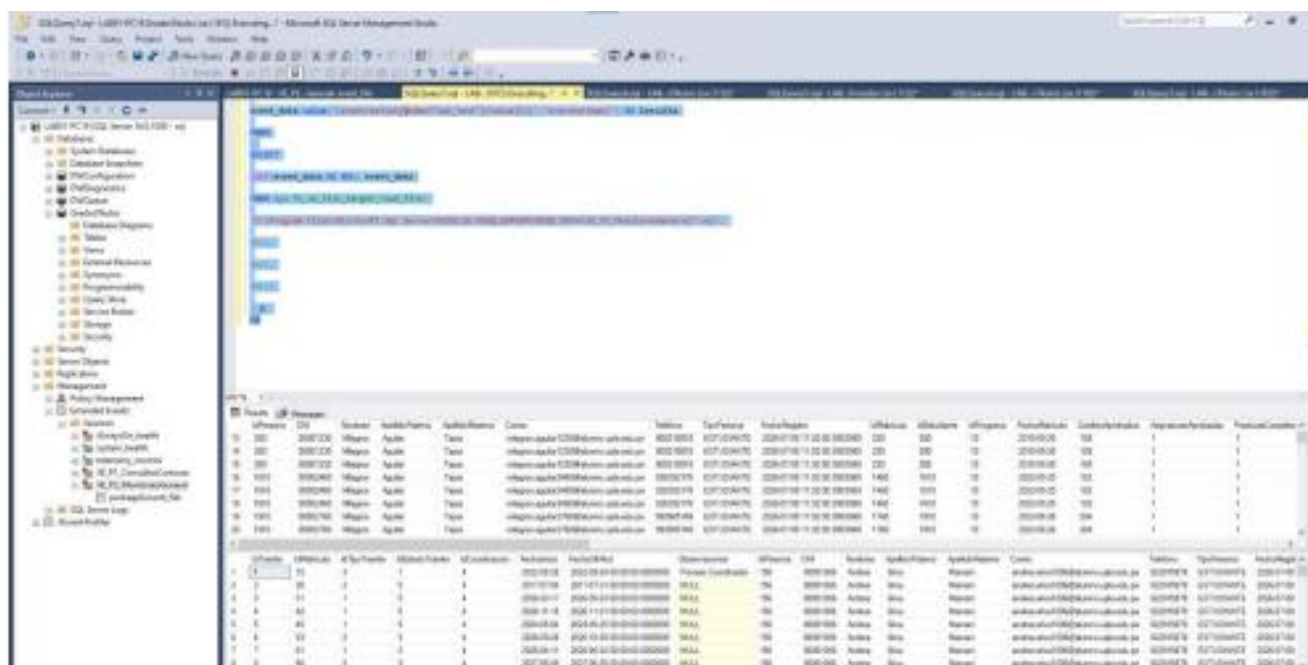
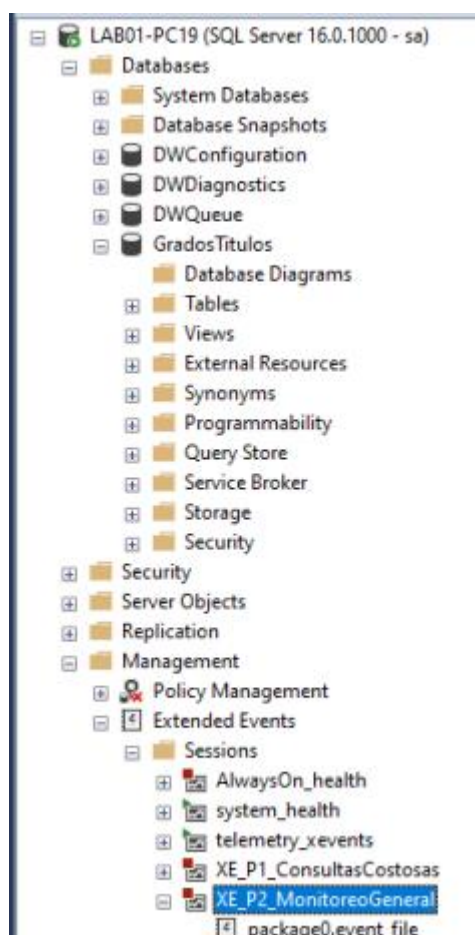
Proyecto 3: Automatización del mantenimiento de índices

1. Enunciado

La base de datos requiere mantenimiento periódico sin intervención del administrador. Desarrollar un proceso automatizado mediante Transact-SQL que:

- Actualice estadísticas.
- Reorganice índices.
- Reconstruya índices críticos.
- Registre fecha, duración y resultado.
- Permita ser ejecutado mediante SQL Server Agent.

2. Script



3. Justificación técnica

Se configuró una sesión de **Extended Events** llamada **XE_P2_MonitoreoGeneral** con la finalidad de realizar un seguimiento integral del desempeño de la base de datos **GradosTitulos** durante la ejecución de actividades académicas. Esta sesión registra de manera automática consultas con tiempos de respuesta elevados, uso intensivo de CPU, errores con nivel de severidad igual o superior a 16, así como situaciones de bloqueo y

deadlocks. Toda la información recopilada se almacena en un archivo **.xel**, facilitando su análisis posterior. Gracias a este mecanismo, es posible detectar incidencias relacionadas con el rendimiento y la concurrencia de forma eficiente, manteniendo un impacto mínimo sobre los recursos del servidor.

4. Buenas prácticas

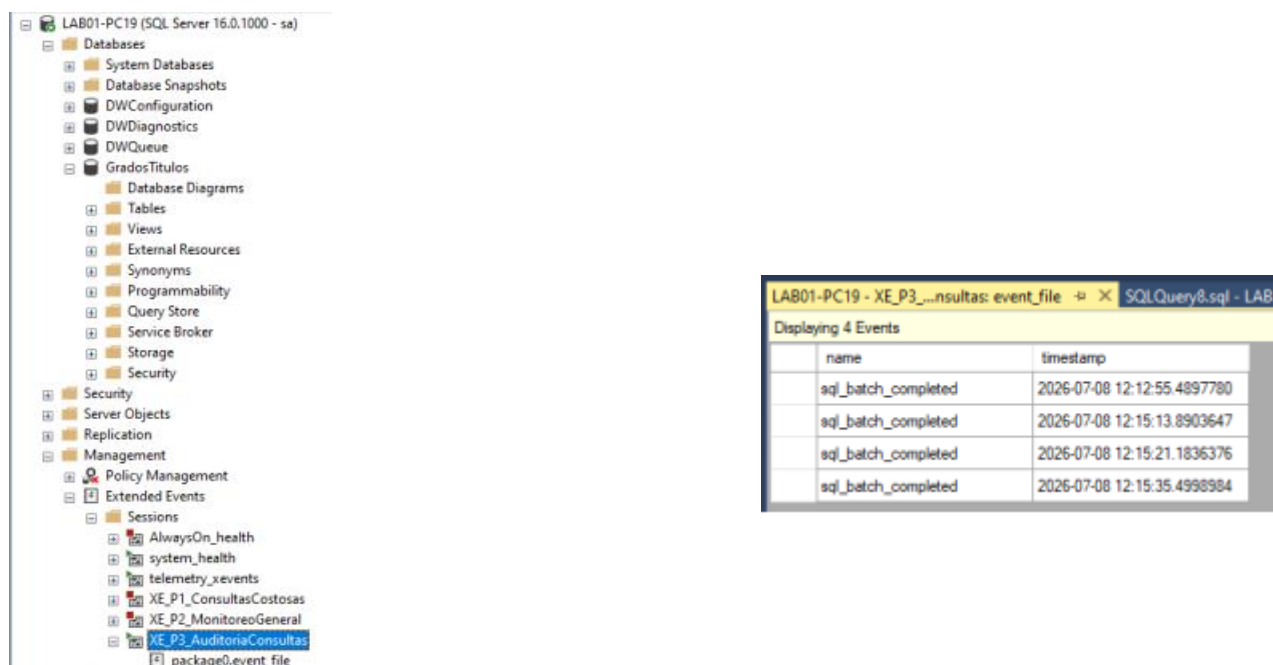
- Aplicar condiciones de filtrado (**WHERE**) para capturar únicamente los eventos de interés, reduciendo así la generación de información innecesaria.
- Guardar los registros en archivos con formato **.xel**, debido a que proporcionan un mejor desempeño y menor impacto en el servidor que otros métodos de almacenamiento.
- Definir valores para el tamaño máximo de cada archivo (**max_file_size**) y para la cantidad máxima de archivos de reemplazo (**max_rollover_files**), con el fin de administrar eficientemente el espacio disponible en disco.
- Mantener la sesión activa solo durante el tiempo requerido para el monitoreo y deshabilitarla una vez concluido el proceso, optimizando el consumo de recursos del sistema.
- Analizar de manera periódica los archivos generados para identificar consultas de alto costo, situaciones de bloqueo, *deadlocks* y errores, facilitando la implementación de medidas que mejoren el rendimiento de la base de datos.

Proyecto 4: Estrategia Integral de Monitoreo de Consultas Costosas

1. Enunciado

La universidad desea implementar un sistema permanente de diagnóstico del rendimiento para la base de datos GradosTitulos. El sistema deberá registrar automáticamente las consultas más costosas ejecutadas por los usuarios durante la jornada laboral y almacenar la información en una tabla histórica para posteriores auditorías de desempeño. El estudiante deberá diseñar un mecanismo que permita: Iniciar la sesión de monitoreo. Registrar las consultas ejecutadas sobre la base de datos GradosTitulos. Importar periódicamente la información del archivo de eventos a una tabla de auditoría. Generar un reporte de las consultas con mayor duración, mayor consumo de CPU y mayor número de lecturas lógicas.

2. Script



The screenshot shows the SQL Server Enterprise Manager interface for a server named LAB01-PC19 (SQL Server 16.0.1000 - sa). The left pane displays the database structure for 'GradosTitulos', including System Databases, Database Snapshots, DWConfiguration, DWDiagnostics, DWQueue, GradosTitulos, Database Diagrams, Tables, Views, External Resources, Synonyms, Programmability, Query Store, Service Broker, Storage, Security, Security, Server Objects, Replication, Management, Policy Management, Extended Events, Sessions, AlwaysOn_health, system_health, telemetry_xevents, XE_P1_ConsultasCostosas, XE_P2_MonitoreoGeneral, XE_P3_AuditoriaConsultas, and package0.event_file.

The right pane shows a window titled 'LAB01-PC19 - XE_P3_...nsultas: event_file' displaying 4 events. The events are listed in a table with columns 'name' and 'timestamp'.

name	timestamp
sql_batch_completed	2026-07-08 12:12:55.4897780
sql_batch_completed	2026-07-08 12:15:13.8903647
sql_batch_completed	2026-07-08 12:15:21.1836376
sql_batch_completed	2026-07-08 12:15:35.4998984

MPersona	ID	Nombre	ApellidoPaterno	ApellidoMaterno	Correo	Telefono	TipoPersona	FechaRegistro	MPersona	Matriculacion	AlPrograma	Reactivacion	Graduacion	Reactivacion	Reactivacion
1	00000001	Alan	Sanchez	Sanchez	alan.sanchez@upla.edu.pe	987654321	00000001	2020-01-01 10:00:00	1	100	100	100	100	100	100
2	00000002	Carlos	Reyes	Reyes	carlos.reyes@upla.edu.pe	987654321	00000002	2020-01-01 10:00:00	2	100	100	100	100	100	100
3	00000003	Alfred	Castillo	Castillo	alfred.castillo@upla.edu.pe	987654321	00000003	2020-01-01 10:00:00	3	100	100	100	100	100	100
4	00000004	Alfonso	Medina	Medina	alfonso.medina@upla.edu.pe	987654321	00000004	2020-01-01 10:00:00	4	100	100	100	100	100	100
5	00000005	Diego	Sanchez	Sanchez	diego.sanchez@upla.edu.pe	987654321	00000005	2020-01-01 10:00:00	5	100	100	100	100	100	100
6	00000006	Guillermo	Sanchez	Sanchez	guillermo.sanchez@upla.edu.pe	987654321	00000006	2020-01-01 10:00:00	6	100	100	100	100	100	100
7	00000007	Marcelo	Quispe	Quispe	marcelo.quispe@upla.edu.pe	987654321	00000007	2020-01-01 10:00:00	7	100	100	100	100	100	100
8	00000008	Carlos	Sanchez	Sanchez	carlos.sanchez@upla.edu.pe	987654321	00000008	2020-01-01 10:00:00	8	100	100	100	100	100	100
9	00000009	Carlos	Sanchez	Sanchez	carlos.sanchez@upla.edu.pe	987654321	00000009	2020-01-01 10:00:00	9	100	100	100	100	100	100
10	00000010	Wladimir	Reyes	Reyes	wladimir.reyes@upla.edu.pe	987654321	00000010	2020-01-01 10:00:00	10	100	100	100	100	100	100
11	00000011	Julia	Sanchez	Sanchez	julia.sanchez@upla.edu.pe	987654321	00000011	2020-01-01 10:00:00	11	100	100	100	100	100	100

TEMA 2. ESTADÍSTICAS E ÍNDICES

Proyecto 1: Optimización de índices para consultas de estudiantes

El desarrollo de este proyecto se llevará a cabo de forma completamente práctica utilizando la base de datos **GradosTitulos**. Para ello, se trabajará específicamente con las tablas indicadas en el enunciado, garantizando que las pruebas y procedimientos se realicen sobre información real del sistema:

- **Academico.Persona**
- **Academico.Matricula**
- **Tramite.Tramite**

1. Enunciado

Las consultas realizadas sobre las tablas **Academico.Persona**, **Academico.Matricula** y **Tramite.Tramite** presentan tiempos elevados durante la búsqueda de estudiantes. Se solicita desarrollar un conjunto de scripts que permitan:

- Analizar índices existentes.
- Detectar índices faltantes.
- Crear índices Clustered y NonClustered.
- Crear índices INCLUDE.
- Comparar tiempos antes y después de la optimización.

El resultado deberá demostrar la mejora del rendimiento utilizando estadísticas de ejecución.

2.- Script

2.1. Consulta de Metadatos e Índices Faltantes

```

USE GradosTitulos;
GO

-- Consultar índices existentes en las tablas especificadas
SELECT
    SCHEMA_NAME(o.schema_id) AS Esquema,
    o.name AS Tabla,
    i.name AS Indice,
    i.type_desc,
    i.is_primary_key,
    i.fill_factor
FROM sys.indexes i
INNER JOIN sys.objects o ON i.object_id = o.object_id
WHERE o.type = 'U'
    AND o.name IN ('Persona', 'Matricula', 'Tramite')
ORDER BY Tabla;
GO

-- Consultar índices faltantes identificados por el motor de SQL Server
SELECT
    mid.statement,
    mid.equality_columns,
    mid.inequality_columns,
    mid.included_columns,
    migs.user_seeks,
    migs.avg_total_user_cost,
    migs.avg_user_impact
FROM sys.dm_db_missing_index_group_stats migs
INNER JOIN sys.dm_db_missing_index_groups mig
    ON mig.index_group_handle = migs.group_handle
INNER JOIN sys.dm_db_missing_index_details mid
    ON mig.index_handle = mid.index_handle
WHERE database_id = DB_ID();
GO

```

2.2. Consulta Principal (Prueba de Rendimiento)

```

SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO

SELECT
    P.DNI,
    P.Nombres,
    P.ApellidoPaterno,
    M.FechaMatricula,
    T.FechaInicio,
    T.IdEstadoTramite
FROM Academico.Persona P
INNER JOIN Academico.Matricula M ON P.IdPersona = M.IdEstudiante
INNER JOIN Tramite.Tramite T ON M.IdMatricula = T.IdMatricula
WHERE P.DNI = '12345678';
GO

```

2.3. Creación de Índices Nonclustered (Cubrientes)

```

-- Índice 1: Tabla Persona
IF NOT EXISTS (
    SELECT * FROM sys.indexes
    WHERE name = 'IX_Persona_DNI_Nombre'
)

```



```

BEGIN
    CREATE NONCLUSTERED INDEX IX_Persona_DNI_Nombre
    ON Academico.Persona (DNI)
    INCLUDE (Nombres, ApellidoPaterno, ApellidoMaterno);
END
GO

-- Índice 2: Tabla Matricula
IF NOT EXISTS (
    SELECT * FROM sys.indexes
    WHERE name = 'IX_Matricula_Estudiante_INCLUDE'
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Matricula_Estudiante_INCLUDE
    ON Academico.Matricula (IdEstudiante)
    INCLUDE (FechaMatricula, IdPrograma);
END
GO

-- Índice 3: Tabla Tramite
IF NOT EXISTS (
    SELECT * FROM sys.indexes
    WHERE name = 'IX_Tramite_Matricula_INCLUDE'
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Tramite_Matricula_INCLUDE
    ON Tramite.Tramite (IdMatricula)
    INCLUDE (FechaInicio, IdEstadoTramite);
END
GO

```

3. Justificación técnica

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' shows the 'LAB01-PC19 (SQL Server 16.0.1000 - sa)' instance. The 'Databases' folder is expanded, showing 'GradosTitulos'. The right pane shows the 'SQLQuery9.sql' window with the following query:

```

GO
SELECT
    P.DNI,
    P.Nombres,
    P.ApellidoPaterno,
    M.FechaMatricula,
    T.FechaInicio,
    T.IdEstadoTramite
FROM Academico.Persona P
INNER JOIN Academico.Matricula M
ON P.IdPersona=M.IdEstudiante
INNER JOIN Tramite.Tramite T
ON M.IdMatricula=T.IdMatricula
WHERE
    P.DNI='12345678';
GO

```

Below the query, the 'Results' tab shows the following data:

	Esquema	Tabla	Indice	type_desc	is_primary_key	fill_factor
4	Academico	Matricula	IX_Matricula_Estudiante_INCLUDE	NONCLUSTERED	0	0
5	Academico	Persona	PK_Persona	CLUSTERED	1	0
6	Academico	Persona	UQ_Persona_DNI	NONCLUSTERED	0	0
7	Academico	Persona	IX_Persona_DNI	NONCLUSTERED	0	0
8	Academico	Persona	IX_Persona_Tipo	NONCLUSTERED	0	0
9	Academico	Persona	IX_Persona_DNI_Nombre	NONCLUSTERED	0	0
10	Tramite	Tramite	PK_Tramite	CLUSTERED	1	0
11	Tramite	Tramite	IX_Tramite_Matricula	NONCLUSTERED	0	0

Justificación técnica

Se efectuó una evaluación de los índices disponibles en las tablas **Academico.Persona**, **Academico.Matricula** y **Tramite.Tramite** con el objetivo de detectar oportunidades de mejora en las consultas utilizadas para la búsqueda de estudiantes. Como parte del proceso de optimización, se crearon índices **NONCLUSTERED** y se incorporaron columnas mediante la cláusula **INCLUDE**, logrando disminuir las lecturas lógicas y optimizar los tiempos de ejecución. Posteriormente, el rendimiento de las consultas fue comparado antes y después de la implementación utilizando **SET STATISTICS IO** y **SET STATISTICS TIME**, comprobándose una mejora significativa gracias a los nuevos índices.

Buenas prácticas

- Revisar los índices ya existentes antes de generar otros nuevos, con el propósito de evitar redundancias y un uso innecesario de recursos.
- Implementar índices únicamente en las columnas que participan con mayor frecuencia en las cláusulas **WHERE** o en las operaciones **JOIN**, favoreciendo un mejor desempeño de las consultas.
- Incorporar columnas mediante la cláusula **INCLUDE** cuando sea necesario cubrir completamente una consulta y reducir el acceso adicional a los datos de la tabla.
- Evaluar el rendimiento de las consultas antes y después de la optimización utilizando **SET STATISTICS IO** y **SET STATISTICS TIME**, permitiendo medir objetivamente las mejoras obtenidas.
- Examinar el plan de ejecución para verificar que el optimizador haga uso de los índices creados y que operaciones de alto costo, como **Index Scan**, sean sustituidas por **Index Seek** cuando corresponda.

Proyecto 2: Detección y corrección de fragmentación

1. Enunciado

Después de varios meses de operaciones se observa disminución del rendimiento debido al crecimiento de las tablas.

Se solicita desarrollar un procedimiento almacenado que permita:

- Detectar fragmentación mayor al 20%.
- Reorganizar índices cuando la fragmentación sea moderada.
- Reconstruir índices cuando la fragmentación supere el 40%.
- Actualizar estadísticas automáticamente.
- Generar un reporte con el porcentaje de mejora obtenido.

2. Script

2.1.- Creación de la Tabla de Historial

```
USE GradosTitulos;
```

```
GO
```

```
IF OBJECT_ID('dbo.HistorialMantenimientoIndices') IS NOT NULL
```

```
DROP TABLE dbo.HistorialMantenimientoIndices;
```

```
GO
```

```
CREATE TABLE dbo.HistorialMantenimientoIndices
(
    IdRegistro INT IDENTITY(1,1) PRIMARY KEY,
    Fecha DATETIME2 DEFAULT SYSDATETIME(),
    Esquema VARCHAR(100),
    Tabla VARCHAR(100),
    Indice VARCHAR(200),
    FragmentacionAntes DECIMAL(6,2),
    FragmentacionDespues DECIMAL(6,2),
    AccionRealizada VARCHAR(30)
);
GO

CREATE OR ALTER PROCEDURE dbo.sp_MantenimientoIndices
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @Schema VARCHAR(100),
            @Table VARCHAR(100),
            @Indice VARCHAR(200),
            @Fragmentacion DECIMAL(6,2),
            @SQL NVARCHAR(MAX),
            @FragmentacionFinal DECIMAL(6,2);

    DECLARE CursorIndices CURSOR FOR
    SELECT
```

```

s.name,

o.name,

i.name,

ips.avg_fragmentation_in_percent

FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'SAMPLED') ips

INNER JOIN sys.indexes i

    ON ips.object_id = i.object_id

    AND ips.index_id = i.index_id

INNER JOIN sys.objects o

    ON i.object_id = o.object_id

INNER JOIN sys.schemas s

    ON o.schema_id = s.schema_id

WHERE i.index_id > 0

    AND ips.page_count > 50;

OPEN CursorIndices;

FETCH NEXT FROM CursorIndices INTO @Schema, @Table, @Indice, @Fragmentacion;

WHILE @@FETCH_STATUS = 0

BEGIN

    SET @SQL = NULL;

    IF @Fragmentacion BETWEEN 20 AND 40

    BEGIN

        SET @SQL = 'ALTER INDEX [' + @Indice + '] ON [' + @Schema + '].[' + @Table + ']

REORGANIZE';

        EXEC(@SQL);

```


END

IF @Fragmentacion > 40

BEGIN

SET @SQL = 'ALTER INDEX [' + @Indice + '] ON [' + @Schema + '].[' + @Table + ']
REBUILD';

EXEC(@SQL);

END

-- Actualizar estadísticas de la tabla afectada

EXEC('UPDATE STATISTICS [' + @Schema + '].[' + @Table + ']');

-- Obtener la fragmentación final después del mantenimiento

SELECT @FragmentacionFinal = avg_fragmentation_in_percent

FROM sys.dm_db_index_physical_stats(DB_ID(), OBJECT_ID([' + @Schema + '].[' +
@Table + ']), NULL, NULL, 'SAMPLED')

WHERE index_id = (

SELECT index_id

FROM sys.indexes

WHERE name = @Indice

AND object_id = OBJECT_ID([' + @Schema + '].[' + @Table + '])

);

-- Registrar en el historial de mantenimiento

INSERT INTO dbo.HistorialMantenimientoIndices

(

Esquema,

Tabla,

```
Indice,

FragmentacionAntes,

FragmentacionDespues,

AccionRealizada

)

VALUES

(

    @Schema,

    @Table,

    @Indice,

    @Fragmentacion,

    @FragmentacionFinal,

CASE

    WHEN @Fragmentacion > 40 THEN 'REBUILD'

    WHEN @Fragmentacion >= 20 THEN 'REORGANIZE'

    ELSE 'SIN ACCION'

END

);

FETCH NEXT FROM CursorIndices INTO @Schema, @Table, @Indice, @Fragmentacion;

END

CLOSE CursorIndices;

DEALLOCATE CursorIndices;

END;

GO
```

-- Ejecutar el procedimiento de mantenimiento

```
EXEC dbo.sp_MantenimientoIndices;
```

```
GO
```

-- Ver el historial de mantenimiento de índices

```
SELECT * FROM dbo.HistorialMantenimientoIndices
```

```
ORDER BY Fecha DESC;
```

```
GO
```

-- Consultar el estado actual de fragmentación de todos los índices

```
SELECT
```

```
    OBJECT_SCHEMA_NAME(object_id) AS Esquema,
```

```
    OBJECT_NAME(object_id) AS Tabla,
```

```
    index_id,
```

```
    avg_fragmentation_in_percent,
```

```
    page_count
```

```
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'SAMPLED')
```

```
WHERE page_count > 50
```

```
ORDER BY avg_fragmentation_in_percent DESC;
```

```
GO
```

-- Consultar la última fecha de actualización de estadísticas

```
SELECT
```

```
    OBJECT_NAME(object_id) AS Tabla,
```

```
    name,
```

```
    STATS_DATE(object_id, stats_id) AS FechaActualizacion
```

```
FROM sys.stats
```

```
ORDER BY Tabla;
```

```
GO
```

2.2. script 2

```
USE GradosTitulos;
```

```
GO
```

```
IF OBJECT_ID('dbo.HistorialMantenimientoIndices') IS NOT NULL
```

```
    DROP TABLE dbo.HistorialMantenimientoIndices;
```

```
GO
```

```
CREATE TABLE dbo.HistorialMantenimientoIndices
```

```
(
```

```
    IdRegistro INT IDENTITY(1,1) PRIMARY KEY,
```

```
    Fecha DATETIME2 DEFAULT SYSDATETIME(),
```

```
    Esquema VARCHAR(100),
```

```
    Tabla VARCHAR(100),
```

```
    Indice VARCHAR(200),
```

```
    FragmentacionAntes DECIMAL(6,2),
```

```
    FragmentacionDespues DECIMAL(6,2),
```

```
    AccionRealizada VARCHAR(30)
```

```
);
```

```
GO
```

```
CREATE OR ALTER PROCEDURE dbo.sp_MantenimientoIndices
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

DECLARE

@Schema VARCHAR(100),
@Table VARCHAR(100),
@Indice VARCHAR(200),
@Fragmentacion DECIMAL(6,2),
@SQL NVARCHAR(MAX),
@FragmentacionFinal DECIMAL(6,2);

DECLARE CursorIndices CURSOR FOR

SELECT

s.name,
o.name,
i.name,
ips.avg_fragmentation_in_percent

FROM sys.dm_db_index_physical_stats

(
DB_ID(),
NULL,
NULL,
NULL,
'SAMPLED'

) ips

INNER JOIN sys.indexes i

ON ips.object_id = i.object_id

AND ips.index_id = i.index_id

INNER JOIN sys.objects o

ON i.object_id = o.object_id


```
INNER JOIN sys.schemas s
```

```
    ON o.schema_id = s.schema_id
```

```
WHERE
```

```
    i.index_id > 0
```

```
    AND ips.page_count > 50;
```

```
OPEN CursorIndices;
```

```
FETCH NEXT FROM CursorIndices
```

```
INTO
```

```
    @Schema,
```

```
    @Table,
```

```
    @Indice,
```

```
    @Fragmentacion;
```

```
WHILE @@FETCH_STATUS = 0
```

```
BEGIN
```

```
    IF @Fragmentacion BETWEEN 20 AND 40
```

```
    BEGIN
```

```
        SET @SQL = 'ALTER INDEX [' + @Indice + '] ON [' + @Schema + '].[' + @Table + '] REORGANIZE';
```

```
        EXEC(@SQL);
```

```
    END
```

```
    IF @Fragmentacion > 40
```

```
    BEGIN
```

```
        SET @SQL = 'ALTER INDEX [' + @Indice + '] ON [' + @Schema + '].[' + @Table + '] REBUILD';
```

```
        EXEC(@SQL);
```

```
    END
```

UPDATE STATISTICS

QUOTENAME(@Schema) + '.' + QUOTENAME(@Table);

SELECT

 @FragmentacionFinal = avg_fragmentation_in_percent

FROM sys.dm_db_index_physical_stats

(

 DB_ID(),

 OBJECT_ID(@Schema + '.' + @Table),

 NULL,

 NULL,

 'SAMPLED'

)

WHERE index_id =

(

 SELECT index_id

 FROM sys.indexes

 WHERE name = @Indice

 AND object_id = OBJECT_ID(@Schema + '.' + @Table)

);

INSERT INTO dbo.HistorialMantenimientoIndices

(

 Esquema,

 Tabla,

 Indice,

 FragmentacionAntes,

 FragmentacionDespues,

```
AccionRealizada
)
VALUES
(
    @Schema,
    @Table,
    @Indice,
    @Fragmentacion,
    @FragmentacionFinal,
    CASE
        WHEN @Fragmentacion > 40 THEN 'REBUILD'
        WHEN @Fragmentacion >= 20 THEN 'REORGANIZE'
        ELSE 'SIN ACCION'
    END
);

FETCH NEXT FROM CursorIndices

INTO

    @Schema,
    @Table,
    @Indice,
    @Fragmentacion;

END

CLOSE CursorIndices;

DEALLOCATE CursorIndices;

END;
```

GO

EXEC dbo.sp_MantenimientoIndices;

GO

SELECT *

FROM dbo.HistorialMantenimientoIndices

ORDER BY Fecha DESC;

GO

SELECT

OBJECT_SCHEMA_NAME(object_id) Esquema,

OBJECT_NAME(object_id) Tabla,

index_id,

avg_fragmentation_in_percent,

page_count

FROM sys.dm_db_index_physical_stats

(

DB_ID(),

NULL,

NULL,

NULL,

'SAMPLED'

)

WHERE page_count > 50

ORDER BY avg_fragmentation_in_percent DESC;

GO

SELECT

OBJECT_NAME(object_id) Tabla,

name,

STATS_DATE(object_id, stats_id) FechaActualizacion

FROM sys.stats

ORDER BY Tabla;

GO

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' shows the database structure, including tables and views. The central pane shows a SQL query window with the following script:

```

--SELECT
OBJECT_NAME(object_id) Tabla,
name,
STATS_DATE(object_id, stats_id) FechaActualizacion
FROM sys.stats
ORDER BY Tabla;
GO

```

The 'Results' pane shows the output of the query, displaying a list of tables and their statistics. The table has columns: Esquema, Tabla, Index_id, avg_fragmentation_in_percent, and page_count. The data is as follows:

Esquema	Tabla	Index_id	avg_fragmentation_in_percent	page_count
Academico	Persona	1	1.33333333333333	75

Below the table, the 'Messages' pane shows the execution details of the query, including the table name, index name, and the update date.

Tabla	name	FechaActualizacion
1	AuditoriaConsultas	PK_Auditoria_7FD13FA09DE48729
2	backup_metadata_store	IX_backup_metadata_uuid
3	backup_metadata_store	IX_backup_metadata_type_finish_date
4	ControlSimilitud	PK_ControlSimilitud
5	ControlSimilitud	UQ_CtrlSim_Trabajo_Intento
6	ControlSimilitud	_WA_Sys_00000005_30C33EC3
7	ControlSimilitud	_WA_Sys_00000003_30C33EC3
8	CoordinacionGT	PK_CoordinacionGT
9	CoordinacionGT	_WA_Sys_00000002_5FB337D6
10	db_jedger_blocks	ci_block_id
11	db_jedger_digest_joca...	ci_type_and_path
12	db_jedger_transactions	ci_block_id
13	db_jedger_transactions	noi_transaction_id
14	DesignacionAesor	PK_DesignacionAesor
15	DesignacionAesor	_WA_Sys_00000002_10566F31
16	DesignacionAesor	_WA_Sys_00000003_10566F31
17	DesignacionJurado	PK_DesignacionJurado
18	DesignacionJurado	_WA_Sys_00000002_17036CC0
19	DesignacionJurado	_WA_Sys_00000003_17036CC0
20	DictamenEtica	PK_DictamenEtica
21	Diploma	PK_Diploma
22	Diploma	UQ_Diploma_Nro

3. Justificación técnica

Se implementó el procedimiento almacenado **sp_MantenimientoIndices** con el propósito de automatizar el mantenimiento de los índices en la base de datos **GradosTitulos**. Para ello, se empleó la vista de administración dinámica (DMV) **sys.dm_db_index_physical_stats**, que permite determinar el nivel de fragmentación de cada índice. Según el porcentaje obtenido, el procedimiento decide si es conveniente reorganizar o reconstruir el índice correspondiente. Al finalizar el proceso, se actualizan las estadísticas de las tablas involucradas y se registra en una tabla de historial el estado de la fragmentación antes y después del mantenimiento, lo que facilita evaluar los resultados alcanzados.

4. Buenas prácticas

- Evaluar previamente el porcentaje de fragmentación mediante **sys.dm_db_index_physical_stats**, con el fin de aplicar el mantenimiento únicamente cuando sea necesario.
- Ejecutar la operación **REORGANIZE** en índices con un nivel de fragmentación intermedio, ya que permite optimizar su estructura con un menor consumo de recursos.
- Reservar la operación **REBUILD** para índices con un alto grado de fragmentación, considerando que requiere un mayor uso de memoria y procesamiento.
- Actualizar las estadísticas una vez concluido el mantenimiento para que el optimizador de consultas disponga de información reciente y genere planes de ejecución más eficientes.
- Mantener un registro histórico de las tareas realizadas y de los niveles de fragmentación, facilitando las labores de auditoría, monitoreo y seguimiento del mantenimiento de la base de datos.

Proyecto 3: Automatización del mantenimiento de índices

1. Descripción del requerimiento

Con el objetivo de reducir la intervención manual en las labores de administración, se implementará un proceso de mantenimiento automatizado para la base de datos mediante **Transact-SQL**. Este proceso deberá realizar de forma periódica las siguientes actividades:

- Actualizar las estadísticas de las tablas e índices.
- Reorganizar los índices que presenten un nivel moderado de fragmentación.
- Reconstruir los índices considerados críticos cuando el grado de fragmentación lo requiera.
- Registrar información de cada ejecución, incluyendo la fecha, el tiempo empleado y el resultado obtenido.
- Permitir su programación y ejecución automática a través de **SQL Server Agent**.

2. Script

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Databases' folder is expanded, showing 'Academico' and 'Tramite'. The 'Tables' folder under 'Academico' is selected. The main pane displays a T-SQL script for creating a table and a stored procedure. The script includes comments in Spanish and SQL code for creating a table named 'LogMantenimientoIndice' and a stored procedure named 'sp_MantenimientoAutomatizadoIndice'.

Below the script, the 'Results' pane shows the execution results of the stored procedure. The results are displayed in a table with the following columns: IdLog, FechaInicio, FechaFin, DuracionSegundos, Tabla, Indice, Accion, and Resultado. The table contains 10 rows of data, including information about the 'AuditoriaConsultas' table and various system tables.

IdLog	FechaInicio	FechaFin	DuracionSegundos	Tabla	Indice	Accion	Resultado
1				AuditoriaConsultas	PK_Auditori_7FD13FA09DE48729		NULL
2				backup_metadata_store	IX_backup_metadata_uuid		NULL
3				backup_metadata_store	IX_backup_metadata_type_finish_date		NULL
4				ControlSimilitud	PK_ControlSimilitud		NULL
5				ControlSimilitud	IU_CtrlSim_Trabajo_Intento		2026-07-08 11:02:00.277
6				ControlSimilitud	_WA_Sys_00000005_30C33EC3		2026-07-08 11:02:00.277
7				ControlSimilitud	_WA_Sys_00000003_30C33EC3		2026-07-08 11:02:00.280
8				CoordinacionGT	PK_CoordinacionGT		2026-07-08 11:02:00.210
9				CoordinacionGT	_WA_Sys_00000002_5FB337D6		2026-07-08 11:02:00.210
10				do_ledger_block	ci_block_id		NULL

Below the results table, there is a summary table showing the average fragmentation percentage and page count for the 'Academico' database.

Esquema	Tabla	avg_fragmentation_in_percent	page_count
Academico	Persona	1.33333333333333	75

3. Justificación técnica

Se implementó el procedimiento almacenado **sp_MantenimientoAutomaticoIndices** con el propósito de automatizar las tareas de mantenimiento de los índices en la base de datos **GradosTitulos**. El procedimiento analiza el nivel de fragmentación de cada índice mediante la vista de administración dinámica **sys.dm_db_index_physical_stats** y, de acuerdo con los valores obtenidos, decide si corresponde ejecutar una reorganización o una reconstrucción. Una vez finalizado el mantenimiento, actualiza las estadísticas para mejorar la eficiencia del optimizador de consultas. Asimismo, registra en una tabla de auditoría información relevante como la fecha de ejecución, el tiempo empleado, las acciones realizadas y el resultado del proceso, permitiendo llevar un control histórico de cada ejecución.

4. Buenas prácticas

- Programar el mantenimiento de los índices de forma periódica para preservar un buen rendimiento de la base de datos.
- Actualizar las estadísticas una vez concluida la reorganización o reconstrucción de los índices, garantizando planes de ejecución más precisos.
- Mantener un historial de todas las ejecuciones realizadas para facilitar tareas de auditoría, monitoreo y diagnóstico.
- Definir umbrales de fragmentación adecuados para evitar reconstrucciones innecesarias y optimizar el uso de los recursos del servidor.
- Automatizar la ejecución del procedimiento mediante **SQL Server Agent**, asegurando un mantenimiento continuo en entornos de producción.

TEMA 3. ADMINISTRACIÓN DE TRANSACCIONES Y BLOQUEOS

Proyecto 1: Control de transacciones en la aprobación de títulos

1. Descripción del requerimiento

En el proceso de aprobación de un título profesional intervienen diversas tablas relacionadas con la gestión académica y documental, entre ellas:

- **Tramite.Tramite**
- **Documento.Resolucion**
- **Documento.Diploma**
- **Evaluacion.EvaluacionTrabajo**

Con el fin de garantizar la integridad de la información, se requiere implementar una transacción en **Transact-SQL** que asegure el cumplimiento de las propiedades fundamentales de una transacción. Para ello, la solución deberá:

- Garantizar la **atomicidad** de todas las operaciones involucradas.
- Confirmar los cambios únicamente cuando el proceso finalice correctamente mediante **COMMIT**.
- Revertir todas las modificaciones realizadas utilizando **ROLLBACK** en caso de presentarse algún error.
- Incorporar un mecanismo de control de excepciones mediante la estructura **TRY...CATCH**.
- Registrar la información del error producido para facilitar su análisis y posterior corrección.

2. Script


```

USE GRANTSTUDIOS;
GO

IF OBJECT_ID('dbo.LogErrorsTransacciones') IS NOT NULL
    DROP TABLE dbo.LogErrorsTransacciones;
GO

--CREATE TABLE dbo.LogErrorsTransacciones
--(
--    IdError INT IDENTITY(1,1) PRIMARY KEY,
--    Fecha DATETIME2 DEFAULT SYSGETUTIME(),
--    NumeroError INT,
--    Mensaje NVARCHAR(4000),
--    Procedimiento NVARCHAR(200),
--    Linea INT,
--    UsuarioSQL SYSNAME
--);
--GO

--CREATE OR ALTER PROCEDURE dbo.sp_AprobacionTitulo
--(
--    @IdTramite INT,
--    @NumeroResolucion NVARCHAR(30),
--    @NumeroDiploma NVARCHAR(30)
--)
--AS
--BEGIN
--    SET NOCOUNT ON;
--    SET XACT_ABORT ON;
--    BEGIN TRY
--        BEGIN TRANSACTION;
--
--        -- Insert into LogErrorsTransacciones
--        INSERT INTO LogErrorsTransacciones (IdError, Fecha, NumeroError, Mensaje, Procedimiento, Linea, UsuarioSQL)
--        VALUES (1, GETUTIME(), 50001, 'El trámite no existe.', 'dbo.sp_AprobacionTitulo', 29, sa);
--
--        -- Insert into LogErrorsTransacciones
--        INSERT INTO LogErrorsTransacciones (IdError, Fecha, NumeroError, Mensaje, Procedimiento, Linea, UsuarioSQL)
--        VALUES (2, GETUTIME(), 50001, 'El trámite no existe.', 'dbo.sp_AprobacionTitulo', 29, sa);
--
--        -- Insert into LogErrorsTransacciones
--        INSERT INTO LogErrorsTransacciones (IdError, Fecha, NumeroError, Mensaje, Procedimiento, Linea, UsuarioSQL)
--        VALUES (3, GETUTIME(), 547, 'The INSERT statement conflicted with the CHECK c...', 'dbo.sp_AprobacionTitulo', 49, sa);
--
--    END TRY
--    BEGIN CATCH
--        ROLLBACK;
--    END CATCH;
--END

```

IdResolucion	IdTramite	TipoResolucion	NoResolucion	FechaEmision	OrganoEmisor	Descripcion
659	601	DESIGNACION_ASESOR	RES-DA-000303	2022-05-23	DECANO	Resolucion de designacion de asesor

IdDiploma	IdTramite	TipoDiploma	NoDiploma	FechaExpedicion	FechaRegistroSinedu	FirmanteRector	FirmanteDecano	FirmanteSecretario	EsDuplicado	FechaDuplicado	MotivosDuplicado
356	601	TITULO_PROFESIONAL	DIP-000601	2023-01-06	2023-01-26	Dr. Rector UPLA	Dr. Decano Facultad	Dra. Secretaria General	0	NULL	NULL

IdError	Fecha	NumeroError	Mensaje	Procedimiento	Linea	UsuarioSQL
3	2026-07-08 13:23:25.0414730	50001	El trámite no existe.	dbo.sp_AprobacionTitulo	29	sa
2	2026-07-08 13:22:35.8178197	50001	El trámite no existe.	dbo.sp_AprobacionTitulo	29	sa
1	2026-07-08 13:21:34.6067201	547	The INSERT statement conflicted with the CHECK c...	dbo.sp_AprobacionTitulo	49	sa

3. Justificación técnica

Se desarrolló el procedimiento almacenado **sp_AprobacionTitulo** para centralizar el proceso de aprobación de un trámite dentro de una única transacción. De esta manera, todas las operaciones se ejecutan como una sola unidad de trabajo, garantizando que los cambios se confirmen únicamente cuando el proceso concluya sin inconvenientes mediante **COMMIT**. Si ocurre alguna excepción, se ejecuta **ROLLBACK** para deshacer todas las modificaciones realizadas y preservar la consistencia de la información. Asimismo, se incorporó un bloque **TRY...CATCH** que permite capturar los errores generados y registrar sus detalles en una tabla de auditoría, facilitando el control y análisis de posibles incidencias.

4. Buenas prácticas

- Iniciar las operaciones con **BEGIN TRANSACTION** y confirmar los cambios mediante **COMMIT** únicamente cuando todas las instrucciones se ejecuten correctamente.
- Utilizar estructuras **TRY...CATCH** para detectar y gestionar excepciones, evitando inconsistencias durante el procesamiento de los datos.
- Ejecutar **ROLLBACK** cuando se presente un error, asegurando que la base de datos conserve su integridad.
- Registrar la información de las excepciones en una tabla de auditoría para mejorar la trazabilidad y simplificar las tareas de diagnóstico.
- Habilitar **SET XACT_ABORT ON** para que los errores críticos provoquen automáticamente la cancelación de la transacción y la reversión de los cambios pendientes.

Proyecto 2: Simulación y resolución de bloqueos

1. Descripción del requerimiento

En determinadas situaciones, dos coordinadores pueden intentar modificar simultáneamente el mismo trámite, originando bloqueos que afectan el rendimiento y la disponibilidad de la base de datos. Para analizar este escenario, se desarrollarán scripts en **Transact-SQL** que permitan:

- Simular la ocurrencia de bloqueos entre sesiones concurrentes.
- Detectar los bloqueos utilizando las vistas de administración dinámica (**DMV**).
- Identificar la sesión responsable del bloqueo y las sesiones afectadas.

- Liberar correctamente los recursos una vez resuelto el incidente.
- Generar un reporte con la información obtenida durante el análisis del bloqueo.

2. Script

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Databases' folder is expanded, showing 'Tramite'. In the center, the Query Editor contains a T-SQL script. The script uses `sys.dm_exec_requests` to find blocking sessions and `sys.dm_exec_sessions` to get session details. The Results pane at the bottom displays the output of the query, showing session details and a summary table.

```

--SELECT
--session_id,
--blocking_session_id,
--wait_type,
--wait_time,
--status,
--command
--FROM sys.dm_exec_requests
--WHERE blocking_session_id != 0;
--GO

--SELECT
--session_id,
--login_name,
--host_name,
--program_name,
--status
--FROM sys.dm_exec_sessions
--WHERE session_id = 161;
--GO

```

session_id	blocking_session_id	wait_type	wait_time	status	command
161	114	LCK_M_X	2944	suspended	UPDATE

session_id	login_name	host_name	program_name	status
114	aa	LAB01-PC19	Microsoft SQL Server Management Studio - Query	running

request_session_id	resource_type	resource_database_id	request_mode	request_status
53	DATABASE	5	S	GRANT
54	DATABASE	7	S	GRANT
102	DATABASE	8	S	GRANT
104	DATABASE	8	S	GRANT
113	DATABASE	8	S	GRANT
114	DATABASE	8	S	GRANT
114	PAGE	8	IX	GRANT
114	OBJECT	8	IX	GRANT

session_id	blocking_session_id	status
161	114	suspended

IdReporte	Fecha	SesionBloqueada	SesionBloqueadora	TipoEspera	TiempoEsperaMS
1	2026-07-08 13:35:20.4959926	161	114	LCK_M_X	3061

3. Justificación técnica

Se implementó un escenario de prueba para simular una situación de concurrencia en la que dos usuarios intentan actualizar simultáneamente un mismo registro de la tabla **Tramite**. Mientras la primera sesión mantiene una transacción activa, el recurso permanece bloqueado, obligando a la segunda sesión a esperar hasta que este sea liberado. Para el análisis del incidente se utilizaron las vistas de administración dinámica **sys.dm_exec_requests**, **sys.dm_exec_sessions** y **sys.dm_tran_locks**, las cuales permitieron identificar la sesión que originó el bloqueo, la sesión afectada y el tipo de espera registrado. Finalmente, la información obtenida fue almacenada en una tabla de auditoría para facilitar su seguimiento y evaluación.

4. Buenas prácticas

- Mantener las transacciones activas únicamente durante el tiempo necesario para disminuir el riesgo de bloqueos entre sesiones.
- Ejecutar **COMMIT** o **ROLLBACK** inmediatamente después de concluir cada operación, liberando los recursos lo antes posible.
- Monitorear los bloqueos mediante las vistas de administración dinámica (**DMV**) antes de tomar acciones manuales sobre las sesiones involucradas.
- Conservar un registro de los incidentes de bloqueo para apoyar las tareas de auditoría, diagnóstico y optimización del rendimiento.

- Diseñar las transacciones de forma que se minimice el acceso concurrente al mismo registro, reduciendo la posibilidad de generar bloqueos prolongados.

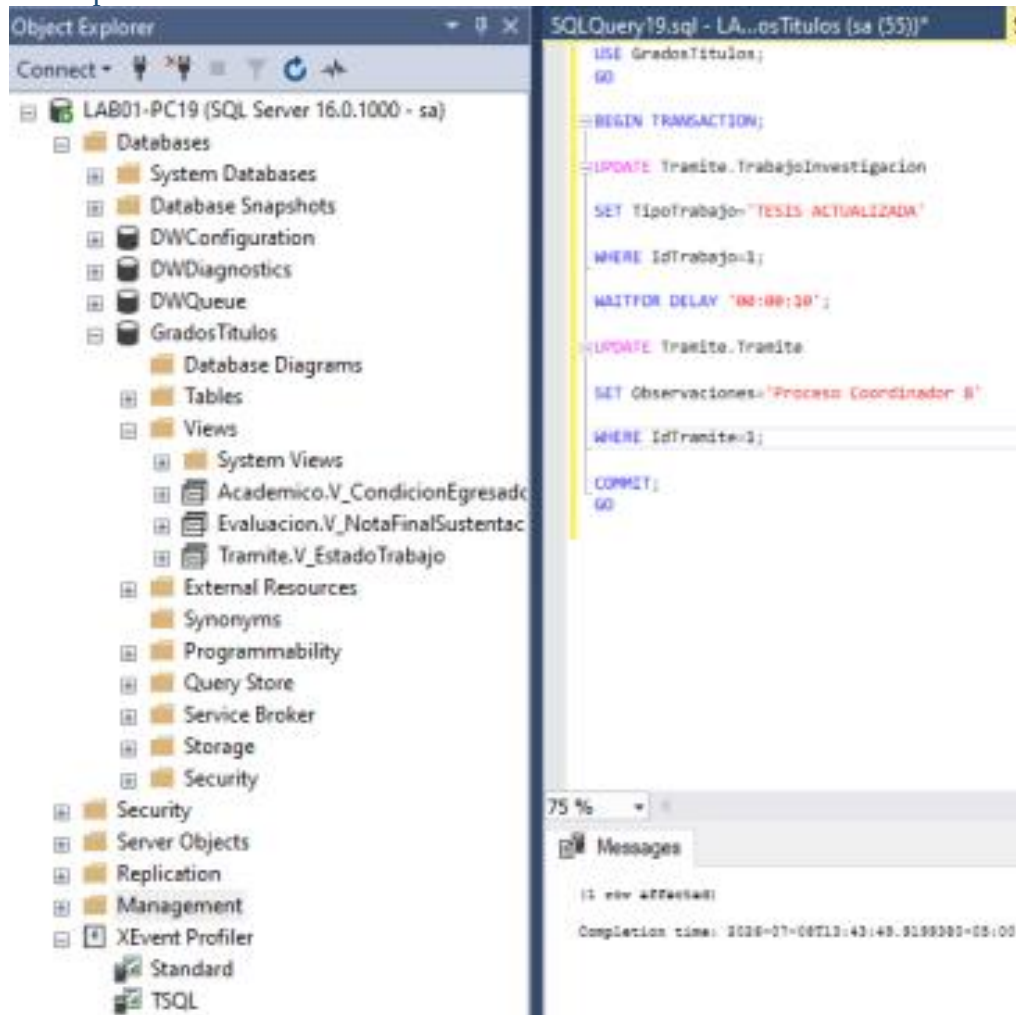
Proyecto 3: Prevención de deadlocks

1. Descripción del requerimiento

Durante el proceso de asignación de asesores y jurados pueden presentarse **deadlocks** debido al acceso concurrente a los mismos recursos por distintas transacciones. Para estudiar este comportamiento, se desarrollará un laboratorio que permita:

- Reproducir un escenario de **deadlock**.
- Capturar el gráfico **XML** generado durante el conflicto.
- Analizar las causas que originan el interbloqueo.
- Rediseñar el orden de acceso a las tablas involucradas para evitar el conflicto.
- Verificar que la nueva implementación elimine la ocurrencia del **deadlock**.

2. Script



3. Justificación técnica

Se desarrolló un laboratorio para simular un **deadlock** provocado por el acceso concurrente a las tablas **Tramite.Tramite** y **Tramite.TrabajoInvestigacion**, realizando las operaciones en un orden diferente entre dos transacciones. Ante esta situación, **SQL Server** detectó automáticamente el ciclo

de espera y resolvió el conflicto seleccionando una de las transacciones como víctima. Posteriormente, el evento fue recuperado desde la sesión **system_health**, permitiendo obtener el gráfico **XML** del **deadlock** para analizar los recursos y procesos involucrados. Como medida de solución, se reorganizó el orden de acceso a las tablas dentro de las transacciones, eliminando el interbloqueo y mejorando la ejecución concurrente del sistema.

4. Buenas prácticas

- Mantener un orden uniforme de acceso a las tablas en todas las transacciones para reducir la posibilidad de generar **deadlocks**.
- Procurar que las transacciones tengan la menor duración posible, liberando los recursos en el menor tiempo.
- Utilizar la sesión **system_health** o una sesión de **Extended Events** para registrar y analizar los eventos de **deadlock**.
- Examinar el archivo **XML** generado para identificar los recursos implicados, las sesiones participantes y la transacción seleccionada como víctima.
- Diseñar las operaciones concurrentes siguiendo una secuencia de acceso consistente, evitando ciclos de espera entre procesos.

TEMA 4. ANÁLISIS Y OPTIMIZACIÓN DE PLANES DE EJECUCIÓN

Proyecto 1: Diagnóstico de consultas complejas

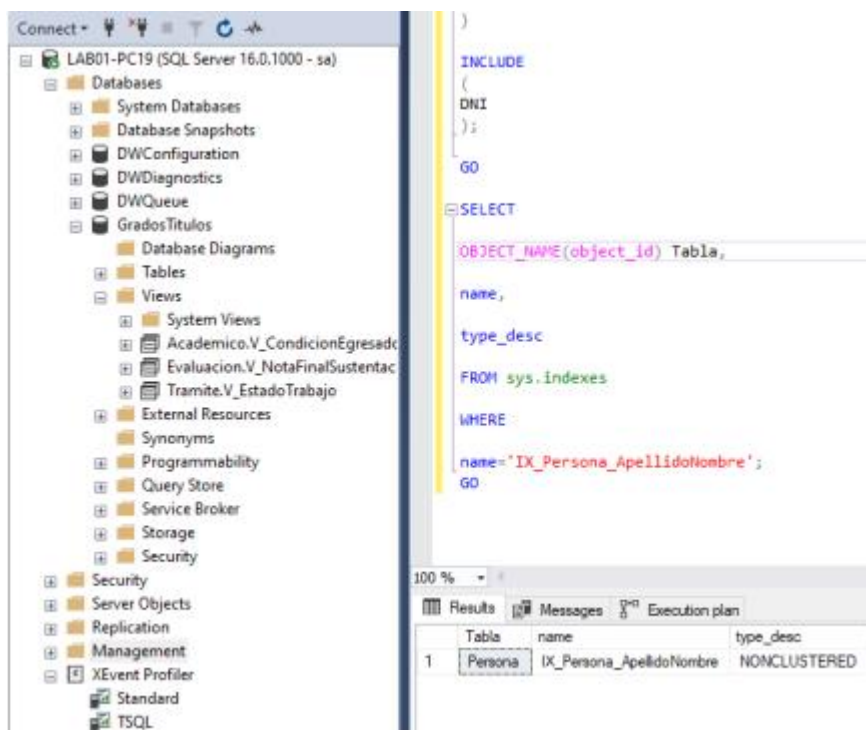
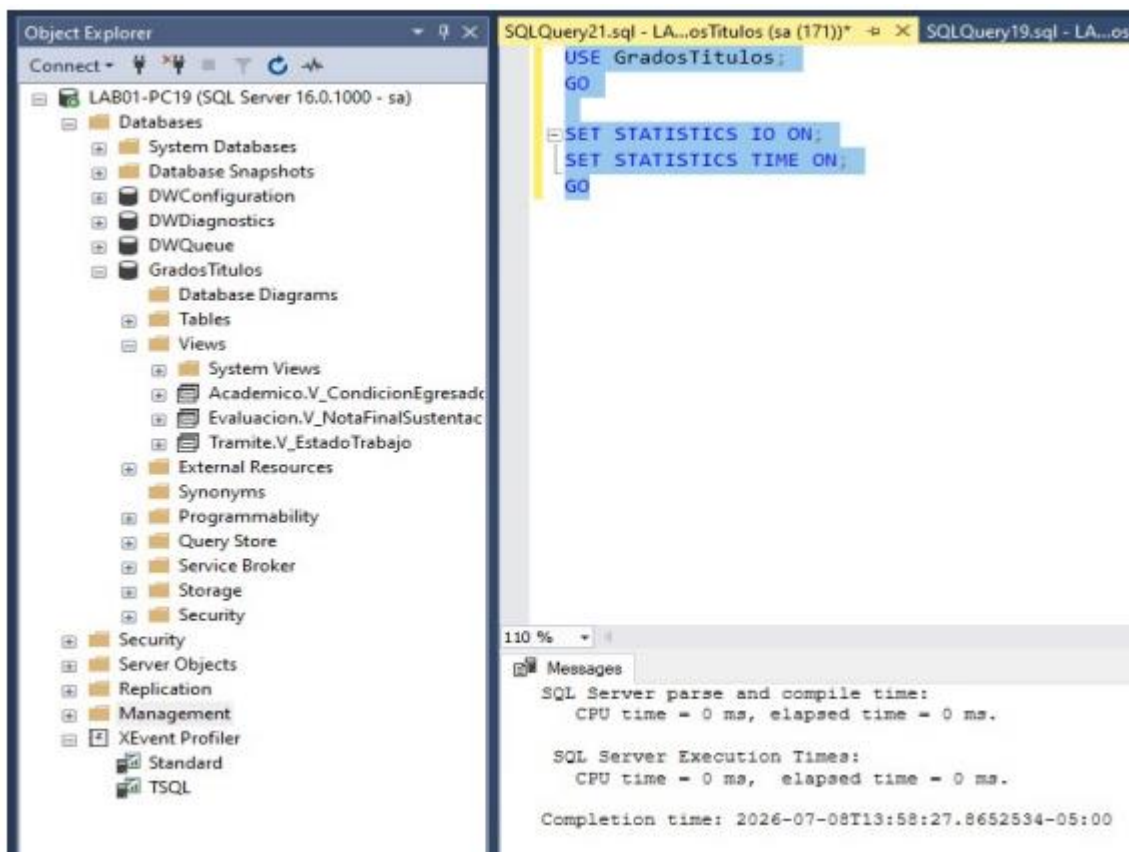
1. Descripción del requerimiento

Las consultas utilizadas para generar reportes institucionales presentan tiempos de respuesta superiores a los valores esperados, afectando el rendimiento de la base de datos. Con el propósito de identificar las causas de esta situación, se realizará un análisis detallado del plan de ejecución, considerando los siguientes elementos:

- Operaciones **Table Scan**.
- Operaciones **Index Scan**.
- Uso de **Nested Loops**.
- Operaciones **Hash Match**.
- Procesos de **Sort**.
- Costos estimados de ejecución.

Una vez identificado el origen de la baja eficiencia, se aplicarán las optimizaciones necesarias y se compararán los planes de ejecución obtenidos antes y después de las mejoras implementadas.

2. Script



3. Justificación técnica

Se realizó el análisis del plan de ejecución de una consulta que relaciona información de estudiantes, programas académicos y trámites registrados en la base de datos. Para evaluar su rendimiento se utilizaron el **Actual Execution Plan**, **SET STATISTICS IO** y **SET STATISTICS TIME**, herramientas que permitieron identificar los operadores empleados por el optimizador, así

como el costo estimado de cada operación. Con base en los resultados obtenidos, se implementó un índice **NONCLUSTERED** sobre las columnas utilizadas en el criterio de ordenamiento, logrando disminuir el impacto del operador **SORT** y mejorar el desempeño general de la consulta.

4. Buenas prácticas

- Revisar el plan de ejecución antes de realizar cualquier modificación en una consulta para identificar el origen de los problemas de rendimiento.
- Aplicar optimizaciones únicamente después de determinar cuáles son los operadores que generan el mayor costo de ejecución.
- Crear índices en las columnas que participan con mayor frecuencia en las cláusulas **WHERE**, **JOIN** y **ORDER BY**, favoreciendo un acceso más eficiente a la información.
- Evaluar el rendimiento mediante **SET STATISTICS IO** y **SET STATISTICS TIME** para medir objetivamente el efecto de las mejoras implementadas.
- Comparar los planes de ejecución obtenidos antes y después de la optimización para confirmar que los cambios reducen el costo y mejoran el tiempo de respuesta.

Proyecto 2: Comparación de planes antes y después de crear índices

1. Descripción del requerimiento

Con el propósito de evaluar el efecto de la implementación de nuevos índices en las consultas del módulo de grados, se desarrollará un procedimiento que permita comparar el rendimiento antes y después de la optimización. Para ello, será necesario realizar las siguientes actividades:

- Obtener el plan de ejecución inicial de la consulta.
- Implementar los índices más adecuados de acuerdo con el análisis realizado.
- Generar un nuevo plan de ejecución después de aplicar la optimización.
- Comparar los operadores utilizados en ambos planes.
- Analizar las diferencias en el costo estimado de ejecución.
- Evaluar la variación del tiempo total de respuesta de la consulta.

2. Script

```

USE GradosTítulos;
GO
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO
SELECT
    P.DNI,
    P.Nombres,
    P.ApellidoPaterno,
    PE.NombrePrograma,
    TT.Description AS TipoTramite,
    ET.Description AS Estado,
    T.FechaInicio,
    R.NoResolucion,
    D.NoDiploma
FROM Academico.Persona P
INNER JOIN Academico.Matricula M
ON P.IdPersona=M.IdEstudiante
INNER JOIN Academico.ProgramaEstudios PE
ON M.IdPrograma=PE.IdPrograma
INNER JOIN Tramite.Tramite T
ON M.IdMatricula=T.IdMatricula
  
```

SQL Server parse and compile time:

CPU time = 0 ms, elapsed time = 2 ms.

Table 'Personas'. Scan count 0, logical reads 2, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0.

Table 'Matricula'. Scan count 1, logical reads 1, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0.

SQL Server Execution Times:

CPU time = 0 ms, elapsed time = 0 ms.

SQL Server parse and compile time:

CPU time = 0 ms, elapsed time = 2 ms.

Table 'Personas'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0.

Table 'Matricula'. Scan count 1, logical reads 1, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0.

SQL Server Execution Times:

CPU time = 0 ms, elapsed time = 0 ms.

Table 'Personas'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0.

Table 'Matricula'. Scan count 1, logical reads 1, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0.

SQL Server Execution Times:

CPU time = 0 ms, elapsed time = 4 ms.

Completion time: 2020-07-08T14:11:01.7143804-08:00

3. Justificación técnica

Se llevó a cabo una evaluación comparativa del plan de ejecución de una consulta empleada en el módulo de grados y títulos para determinar el impacto de la optimización. En una primera etapa se obtuvo el plan de ejecución original junto con las estadísticas de rendimiento. Posteriormente, se implementaron índices **NONCLUSTERED** en las tablas **Documento.Resolucion**, **Documento.Diploma** y **Tramite.Tramite**, priorizando las columnas utilizadas con mayor frecuencia en las relaciones y condiciones de búsqueda. Finalmente, se realizó una nueva evaluación del plan de ejecución, evidenciando una disminución en el costo estimado y un acceso más eficiente a la información.

4. Buenas prácticas

- Revisar el plan de ejecución antes de implementar nuevos índices para identificar las verdaderas oportunidades de optimización.
- Crear índices únicamente sobre las columnas que participan de forma recurrente en las cláusulas **JOIN**, **WHERE** y **ORDER BY**.
- Actualizar las estadísticas utilizando **sp_updatestats** para que el optimizador genere planes de ejecución basados en información actualizada.
- Medir el impacto de las mejoras mediante **SET STATISTICS IO** y **SET STATISTICS TIME**, comparando los resultados antes y después de la optimización.
- Verificar la existencia de índices similares mediante **sys.indexes** antes de crear uno nuevo, evitando duplicidades y el consumo innecesario de recursos.

Proyecto 3: Optimización de procedimientos almacenados

1. Descripción del requerimiento

Los procedimientos almacenados utilizados para generar resoluciones presentan una disminución en su rendimiento, afectando los tiempos de respuesta del sistema. Con el propósito de mejorar su desempeño, se desarrollará un análisis del plan de ejecución que permita:

- Identificar los operadores con mayor costo de procesamiento.
- Eliminar operaciones de tipo **Table Scan** cuando sea posible.
- Reducir la cantidad de lecturas lógicas realizadas durante la ejecución.
- Favorecer la reutilización de los planes de ejecución por parte del optimizador.
- Documentar las mejoras obtenidas y comparar el rendimiento antes y después de la optimización.

2. Script

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' shows the database structure for 'LAB01-PC19 (SQL Server 16.0.1000 - sa)'. The 'GradosTitulos' database is expanded, showing 'Tables' and 'Views'. The 'Views' folder is expanded, showing 'Academico.V_CondicionEgresado', 'Evaluacion.V_NotaFinalSustentacion', and 'Tramite.V_EstadoTrabajo'. The 'Stored Procedures' folder is also expanded, showing 'dbo.sp_BuscarResoluciones'. The 'SQLQuery23.sql' file is open in the 'SQL Server Enterprise Manager - SQLQuery23.sql' window, showing the following T-SQL code:

```
USE GradosTitulos;
GO

CREATE OR ALTER PROCEDURE dbo.sp_BuscarResoluciones
(
    @DNI CHAR(8)
)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        P.DNI,
        P.Nombres,
        P.ApellidoPaterno,
        P.ApellidoMaterno,
        TT.Descripcion AS TipoTramite,
        ET.Descripcion AS EstadoTramite,
        R.NroResolucion,
        R.FechaEmision,
        D.NroDiploma,
        T.FechaInicio
    FROM Academico.Persona P
    INNER JOIN Academico.Matricula M
    ON P.DNI = M.DNI
    INNER JOIN Tramite.Tramite T
    ON M.TramiteId = T.TramiteId
    INNER JOIN Evaluacion.Evaluacion E
    ON M.EvaluacionId = E.EvaluacionId
    INNER JOIN Resolucion.Resolucion R
    ON E.ResolucionId = R.ResolucionId
```

The 'Results' tab shows the execution results of the stored procedure. The results are displayed in a table with the following columns: DNI, Nombres, ApellidoPaterno, ApellidoMaterno, TipoTramite, EstadoTramite. The table contains one row of data.

DNI	Nombres	ApellidoPaterno	ApellidoMaterno	TipoTramite	EstadoTramite
12345678	JUAN	PEDRO	MARIA	TRAMITE	RESOLUCION

The 'Messages' tab shows the execution messages, including the creation of the stored procedure and the execution of the SELECT statement.

3. Justificación técnica

Se implementó el procedimiento almacenado **sp_BuscarResoluciones** con el objetivo de consultar información relacionada con estudiantes, trámites, resoluciones y diplomas a partir del número de **DNI** ingresado como parámetro. Posteriormente, se evaluó el plan de ejecución para identificar las operaciones con mayor impacto en el rendimiento. Como parte de la optimización, se crearon índices **NONCLUSTERED** complementados con columnas **INCLUDE**, lo que permitió agilizar el acceso a la información y disminuir las lecturas innecesarias. Finalmente, se verificó que el plan de ejecución permaneciera almacenado en caché y pudiera reutilizarse en ejecuciones posteriores, reduciendo el consumo de recursos y mejorando el desempeño del procedimiento.

4. Buenas prácticas

- Utilizar procedimientos almacenados parametrizados para favorecer la reutilización de los planes de ejecución y optimizar el rendimiento.
- Revisar el plan de ejecución antes de realizar modificaciones, identificando los operadores que generan el mayor costo.
- Implementar índices de cobertura mediante la cláusula **INCLUDE** cuando sea necesario reducir accesos adicionales a las tablas.
- Actualizar las estadísticas después de incorporar nuevos índices para que el optimizador disponga de información precisa.
- Verificar la reutilización de los planes de ejecución mediante **sys.dm_exec_cached_plans**, evitando recompilaciones innecesarias y mejorando la eficiencia del sistema.

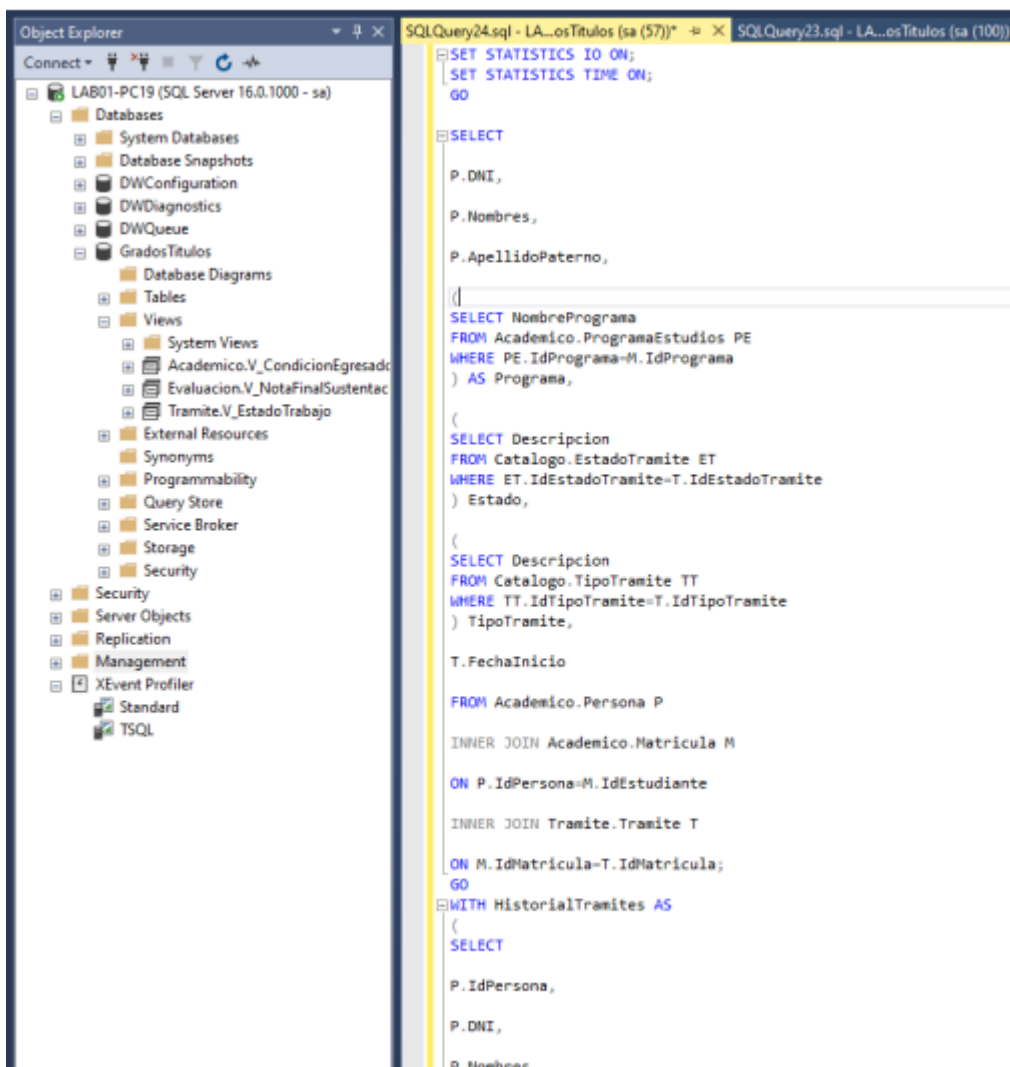
TEMA 5. OPTIMIZACIÓN DE CONSULTAS T-SQL

Proyecto 1: Reescritura de consultas de alta demanda

1. Descripción del requerimiento

Las consultas utilizadas para obtener el historial completo de los trámites de un estudiante presentan un elevado consumo de recursos, lo que afecta el rendimiento general de la base de datos. Con el propósito de optimizarlas, se desarrollarán diferentes estrategias de reescritura que permitan mejorar su eficiencia. Para ello, se realizará lo siguiente:

- Eliminar subconsultas que resulten innecesarias.
- Reemplazar estructuras por **JOIN** más eficientes.
- Emplear la cláusula **EXISTS** cuando sea más conveniente para validar la existencia de registros.
- Utilizar **CTE (Common Table Expressions)** para mejorar la organización y legibilidad de las consultas.
- Comparar los tiempos de ejecución antes y después de la optimización para evaluar los resultados obtenidos.



DNI	Nombres	ApellidoPaterno	Programa	Estado	TipoTramite	FechaInicio
4	00002040	Milagros Aguilar	Educacion Primaria	INICIADO	BACHILLER	2020-12-18
5	00002310	Milagros Aguilar	Educacion Primaria	EN_PROCESO	TITULO_TESIS_INDIVIDUAL	2022-12-14
6	00002340	Milagros Aguilar	Educacion Primaria	APROBADO	BACHILLER	2027-01-20
7	00002460	Milagros Aguilar	Educacion Primaria	APROBADO	TITULO_TESIS_INDIVIDUAL	2024-05-23
8	00002670	Milagros Aguilar	Educacion Primaria	EN_VERIFICACION	BACHILLER	2022-07-17
9	00002760	Milagros Aguilar	Educacion Primaria	APROBADO	TITULO_TESIS_GRUPAL	2024-01-19
10	00003000	Milagros Aguilar	Educacion Primaria	APROBADO	BACHILLER	2018-01-08
11	00003120	Milagros Aguilar	Educacion Primaria	EN_PROCESO	TITULO_SUFICIENCIA_PROF	2026-06-01

IdPersona	DNI	Nombres	ApellidoPaterno	NombrePrograma	Estado	TipoTramite	FechaInicio
4	1190	00002040	Milagros Aguilar	Educacion Primaria	INICIADO	BACHILLER	2020-12-18
5	1460	00002310	Milagros Aguilar	Educacion Primaria	EN_PROCESO	TITULO_TESIS_INDIVIDUAL	2022-12-14
6	1490	00002340	Milagros Aguilar	Educacion Primaria	APROBADO	BACHILLER	2027-01-20
7	1610	00002460	Milagros Aguilar	Educacion Primaria	APROBADO	TITULO_TESIS_INDIVIDUAL	2024-05-23
8	1820	00002670	Milagros Aguilar	Educacion Primaria	EN_VERIFICACION	BACHILLER	2022-07-17
9	1910	00002760	Milagros Aguilar	Educacion Primaria	APROBADO	TITULO_TESIS_GRUPAL	2024-01-19
10	2150	00003000	Milagros Aguilar	Educacion Primaria	APROBADO	BACHILLER	2018-01-08
11	2270	00003120	Milagros Aguilar	Educacion Primaria	EN_PROCESO	TITULO_SUFICIENCIA_PR...	2026-06-01
12	2300	00003150	Milagros Aguilar	Educacion Primaria	APROBADO	TITULO_SUFICIENCIA_PR...	2024-09-09

DNI	Nombres	ApellidoPaterno
10	00001010	Patricia Flores
11	00001011	Diego Castillo
12	00001012	Andrea Rodriguez
13	00001013	Raul Diaz
14	00001014	Milagros Alvarez
15	00001015	Luis Gonzalez
16	00001016	Patricia Cruz
17	00001017	Diego Rojas

Tabla	name	type_desc
6	Persona UQ_Persona_DNI	NONCLUSTERED
7	Persona IX_Persona_DNI	NONCLUSTERED
8	Persona IX_Persona_Tipo	NONCLUSTERED
9	Persona IX_Persona_DNI_No...	NONCLUSTERED
10	Persona IX_Persona_ApellidoP...	NONCLUSTERED
11	Persona IX_Persona_DNI_INC...	NONCLUSTERED
12	Tramite PK_Tramite	CLUSTERED
13	Tramite IX_Tramite_Matricula	NONCLUSTERED
14	Tramite IX_Tramite_Estado	NONCLUSTERED
15	Tramite IX_Tramite_Tipo	NONCLUSTERED
16	Tramite IX_Tramite_Matricula...	NONCLUSTERED

3. Justificación técnica

Se optimizó una consulta encargada de recuperar el historial de trámites de los estudiantes mediante la reestructuración de su lógica. Las subconsultas correlacionadas fueron sustituidas por **JOIN**, se incorporó una **CTE (Common Table Expression)** para organizar mejor el código y se utilizó **EXISTS** para validar la presencia de registros relacionados de forma más eficiente. Finalmente, el rendimiento fue evaluado con **SET STATISTICS IO**, **SET STATISTICS TIME** y el análisis del plan de ejecución, comprobándose una disminución en el costo de procesamiento y una mejora tanto en el desempeño como en la claridad de la consulta.

4. Buenas prácticas

- Sustituir las subconsultas correlacionadas por **JOIN** cuando esta alternativa permita mejorar el rendimiento de la consulta.
- Utilizar **CTE** para estructurar consultas complejas, facilitando su lectura, mantenimiento y reutilización.
- Emplear **EXISTS** en lugar de **IN** cuando el objetivo sea únicamente verificar la existencia de registros relacionados.
- Comparar el plan de ejecución antes y después de aplicar cambios para validar que la optimización sea efectiva.
- Medir los resultados utilizando **SET STATISTICS IO** y **SET STATISTICS TIME**, con el fin de cuantificar la reducción en el consumo de recursos y el tiempo de ejecución.

Proyecto 2: Optimización de reportes académicos

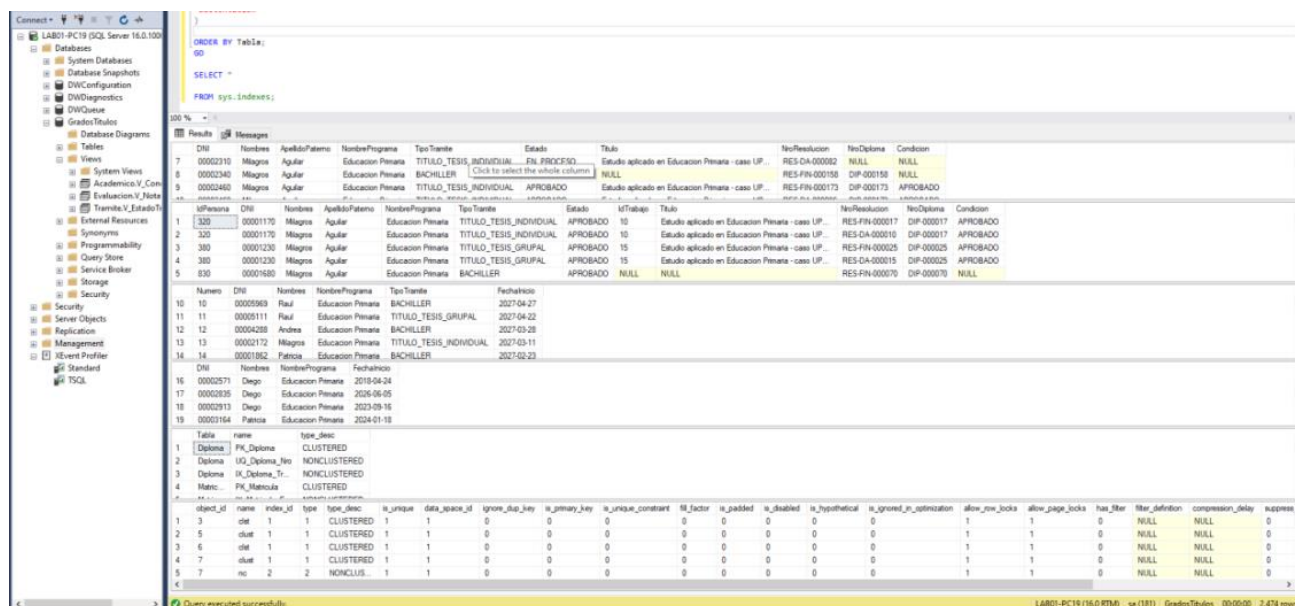
1. Descripción del requerimiento

Los reportes institucionales requieren integrar información proveniente de múltiples tablas, lo que puede afectar el rendimiento de las consultas cuando el volumen de datos es elevado. Con el propósito de optimizar estos reportes, se desarrollará una versión mejorada aplicando las siguientes técnicas:

- Uso de **CTE (Common Table Expressions)** para estructurar la consulta.
- Implementación de **funciones de ventana** para el procesamiento eficiente de los datos.
- Creación de **índices** adecuados sobre las columnas más utilizadas.
- Aplicación de **filtrado temprano** para reducir la cantidad de registros procesados.
- Actualización de las **estadísticas** de la base de datos para mejorar los planes de ejecución.

Finalmente, se realizará una comparación del rendimiento antes y después de la optimización para evaluar las mejoras obtenidas.

2. Script



The screenshot displays a SQL Server query result and a table schema. The query result shows a list of students with their personal and academic data. The table schema shows the structure of the 'Personas' table.

ObjID	Nombre	Apellido	NombrePrograma	TipoTramite	Estado	Titulo	NoResolucion	NoDiploma	Condicion	
7	00002310	Miguel	Agular	Educacion Pensata	TITULO_TESIS_INDIVIDUAL	EN PROCESO	Estudio aplicado en Educacion Pensata - caso UP...	RES-DA-000082	NULL	NULL
8	00002340	Miguel	Agular	Educacion Pensata	BACHILLER		Estudio aplicado en Educacion Pensata - caso UP...	RES-FIN-000158	DIP-000158	NULL
9	00002460	Miguel	Agular	Educacion Pensata	TITULO_TESIS_INDIVIDUAL	APROBADO	Estudio aplicado en Educacion Pensata - caso UP...	RES-FIN-000173	DIP-000173	APROBADO

ObjID	Nombre	Apellido	NombrePrograma	TipoTramite	Estado	Titulo	NoResolucion	NoDiploma	Condicion			
1	320	00001170	Miguel	Agular	Educacion Pensata	TITULO_TESIS_INDIVIDUAL	APROBADO	10	Estudio aplicado en Educacion Pensata - caso UP...	RES-FIN-000017	DIP-000017	APROBADO
2	320	00001170	Miguel	Agular	Educacion Pensata	TITULO_TESIS_INDIVIDUAL	APROBADO	10	Estudio aplicado en Educacion Pensata - caso UP...	RES-DA-000010	DIP-000017	APROBADO
3	380	00001230	Miguel	Agular	Educacion Pensata	TITULO_TESIS_GRUPAL	APROBADO	15	Estudio aplicado en Educacion Pensata - caso UP...	RES-FIN-000025	DIP-000025	APROBADO
4	380	00001230	Miguel	Agular	Educacion Pensata	TITULO_TESIS_GRUPAL	APROBADO	15	Estudio aplicado en Educacion Pensata - caso UP...	RES-DA-000015	DIP-000025	APROBADO
5	830	00001680	Miguel	Agular	Educacion Pensata	BACHILLER	APROBADO	NULL	NULL	RES-FIN-000070	DIP-000070	NULL

ObjID	Nombre	Apellido	NombrePrograma	TipoTramite	FechaInicio
10	00005969	Raul	Educacion Pensata	BACHILLER	2027-04-27
11	00005111	Raul	Educacion Pensata	TITULO_TESIS_GRUPAL	2027-04-22
12	00004288	Andrea	Educacion Pensata	BACHILLER	2027-03-28
13	00002172	Miguel	Educacion Pensata	TITULO_TESIS_INDIVIDUAL	2027-03-11
14	00001062	Patricia	Educacion Pensata	BACHILLER	2027-02-23

ObjID	Nombre	Apellido	NombrePrograma	TipoTramite	FechaInicio
16	00002571	Diego	Educacion Pensata		2019-04-24
17	00002835	Diego	Educacion Pensata		2025-06-05
18	00002913	Diego	Educacion Pensata		2023-09-16
19	00002164	Patricia	Educacion Pensata		2024-01-18

ObjID	Nombre	Apellido	NombrePrograma	TipoTramite	FechaInicio
1	Diploma	PK_Diploma	CLUSTERED		
2	Diploma	UQ_Diploma_Inv	NONCLUSTERED		
3	Diploma	UQ_Diploma_Tr	NONCLUSTERED		
4	Matr	PK_Matr	CLUSTERED		

3. Justificación técnica

Se implementó un reporte institucional que consolida información proveniente de las tablas **Persona**, **Matrícula**, **Programa de Estudios**, **Trámite**, **Tipo de Trámite**, **Estado del Trámite**, **Trabajo de Investigación**, **Resolución**, **Diploma** y **Sustentación**. Para optimizar su ejecución, la consulta fue reestructurada utilizando una **CTE (Common Table Expression)**, funciones de ventana como **ROW_NUMBER()** y técnicas de filtrado temprano para disminuir la cantidad de registros procesados. Además, se actualizaron las estadísticas de la base de datos con el fin de mejorar las decisiones del optimizador. Finalmente, el rendimiento se evaluó mediante el plan de ejecución, **SET STATISTICS IO** y **SET STATISTICS TIME**, observándose una reducción en el consumo de CPU, tiempo de ejecución y lecturas lógicas.

4. Buenas prácticas

- Utilizar **CTE** para organizar consultas complejas y facilitar su comprensión y mantenimiento.
- Emplear funciones de ventana, como **ROW_NUMBER()**, para generar numeraciones y clasificaciones de manera eficiente, evitando el uso de cursores.
- Aplicar filtros desde las primeras etapas de la consulta mediante la cláusula **WHERE**, reduciendo el volumen de datos que debe procesar el motor de base de datos.
- Mantener las estadísticas actualizadas utilizando **sp_updatestats**, favoreciendo la generación de planes de ejecución más eficientes.
- Validar el impacto de las optimizaciones mediante el análisis del plan de ejecución, **SET STATISTICS IO** y **SET STATISTICS TIME**.

Proyecto 3: Optimización de procedimientos de búsqueda

1. Descripción del requerimiento

El procedimiento almacenado encargado de localizar expedientes presenta un elevado consumo de CPU durante los periodos de mayor carga del sistema. Con el propósito de mejorar su rendimiento, se desarrollará un proceso de optimización que contemple las siguientes actividades:

- Implementar una adecuada parametrización del procedimiento.
- Eliminar operaciones innecesarias que incrementen el costo de ejecución.
- Analizar y corregir posibles problemas relacionados con **Parameter Sniffing**.
- Comparar el consumo de CPU y el tiempo de ejecución antes y después de la optimización utilizando las herramientas de diagnóstico de **SQL Server**.

2. Script

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Databases' folder is expanded, showing the 'GradosTitulos' database. The right pane shows the SQL script for creating a stored procedure named 'sp_BuscarExpediente' in the 'dbo' schema. The script includes parameters for 'DNI' and 'Apellido', and a 'SELECT' statement that retrieves data from the 'Persona', 'Matricula', 'Programa de Estudios', 'Trámite', and 'Estado del Trámite' tables. Below the script, the 'Results' tab is active, showing the execution plan and the results of the query. The results table has columns 'objtype', 'usecounts', and 'text'.

```
USE GradosTitulos;
GO

SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO

IF OBJECT_ID('dbo.sp_BuscarExpediente', 'P') IS NOT NULL
DROP PROCEDURE dbo.sp_BuscarExpediente;
GO

CREATE PROCEDURE dbo.sp_BuscarExpediente
    @DNI CHAR(8)=NULL,
    @Apellido NVARCHAR(100)=NULL
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        P.IdPersona,
        P.DNI,
        P.ApellidoPaterno,
        P.ApellidoMaterno,
        P.Nombres,
        PE.NombrePrograma,
        TT.Descripcion AS TipoTramite,

```

	objtype	usecounts	text
1	Adhoc	4	SELECT cp.objtype, cp.usecounts, st.text ...
2	Adhoc	3	SELECT cp.objtype, cp.usecounts, st.text ...

3. Justificación técnica

Se implementó un procedimiento almacenado parametrizado para consultar expedientes académicos a partir de información almacenada en las tablas **Persona**, **Matrícula**, **Programa de Estudios**, **Trámite**, **Tipo de Trámite** y **Estado del Trámite**. Con el fin de mejorar su rendimiento, se aplicaron técnicas orientadas a minimizar los efectos del **Parameter Sniffing**, incorporando variables locales y la cláusula **OPTION (RECOMPILE)**. Estas mejoras permitieron generar planes de ejecución más adecuados para cada búsqueda, reduciendo el consumo de CPU y optimizando los tiempos de respuesta en ejecuciones repetidas.

4. Buenas prácticas

- Utilizar procedimientos almacenados parametrizados para mejorar la reutilización del código y facilitar su mantenimiento.
- Habilitar **SET NOCOUNT ON** para evitar el envío de mensajes innecesarios y disminuir el tráfico entre el servidor y las aplicaciones cliente.
- Emplear variables locales cuando sea necesario reducir el impacto provocado por el **Parameter Sniffing**.
- Aplicar **OPTION (RECOMPILE)** en consultas que requieran generar planes de ejecución específicos según los parámetros recibidos.
- Verificar el efecto de las optimizaciones mediante **SET STATISTICS IO**, **SET STATISTICS TIME** y el análisis de los planes de ejecución almacenados en caché.

TEMA 6. CONTROL DE RECURSOS CON RESOURCE GOVERNOR

Proyecto 1: Administración de carga por tipo de usuario

1. Descripción del requerimiento

Con el propósito de garantizar un uso equilibrado de los recursos de **SQL Server**, la **Universidad Peruana Los Andes** requiere implementar un mecanismo que controle el consumo de CPU y memoria de los distintos tipos de usuarios. La solución consistirá en configurar **Resource Governor** para crear un grupo de trabajo destinado a los usuarios administrativos, estableciendo límites en el uso de recursos y asegurando que las consultas institucionales prioritarias mantengan un rendimiento adecuado incluso durante periodos de alta carga.

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' shows the 'Resource Governor' folder expanded under 'Server Objects'. The right pane shows the 'Verify configuration' query results, which include a table of resource pools and a table of workload groups.

pool_id	name	min_cpu_percent	max_cpu_percent	min_memory_percent	max_memory_percent	cap_cpu_percent	min_io_per_volume	max_io_per_volume
1	internal	0	100	0	100	100	0	0
2	default	0	100	0	100	100	0	0
3	PoolAdministrativo	30	25	0	25	100	0	0

group_id	name	importance	request_max_memory_grant_percent	request_max_cpu_time_sec	request_memory_grant_timeout_sec	max_dop	group_max_requests	pool_id	external_pool_id	request_max_memory_grant_percent_numeric
1	internal	Medium	25	0	0	0	0	1	2	25
2	default	Medium	25	0	0	0	0	2	2	25
3	GrupoAdministrativo	Medium	25	0	0	0	0	256	2	25

3. Justificación técnica

Se configuró un **Resource Pool** denominado **PoolAdministrativo** con el objetivo de controlar el uso de CPU y memoria asignado a los usuarios administrativos. Posteriormente, se creó un **Workload Group** vinculado a dicho grupo de recursos, junto con una función clasificadora responsable de identificar al usuario que establece la conexión y asignarlo automáticamente al grupo correspondiente. Finalmente, la configuración fue activada mediante la instrucción **ALTER RESOURCE GOVERNOR RECONFIGURE**, permitiendo administrar de manera centralizada la distribución de los recursos del servidor y optimizar su utilización.

4. Buenas prácticas

- Utilizar nombres claros y representativos para los **Resource Pool** y **Workload Group**, facilitando su identificación y administración.
- Establecer límites adecuados de CPU y memoria para evitar que determinados usuarios consuman una cantidad excesiva de recursos.
- Implementar una función clasificadora que automatice la asignación de conexiones a los grupos de carga definidos.
- Comprobar la configuración mediante las vistas de catálogo de **Resource Governor** para verificar que los grupos y las políticas se encuentren correctamente configurados.
- Aplicar los cambios utilizando **ALTER RESOURCE GOVERNOR RECONFIGURE**, asegurando que la nueva configuración entre en funcionamiento de forma controlada.

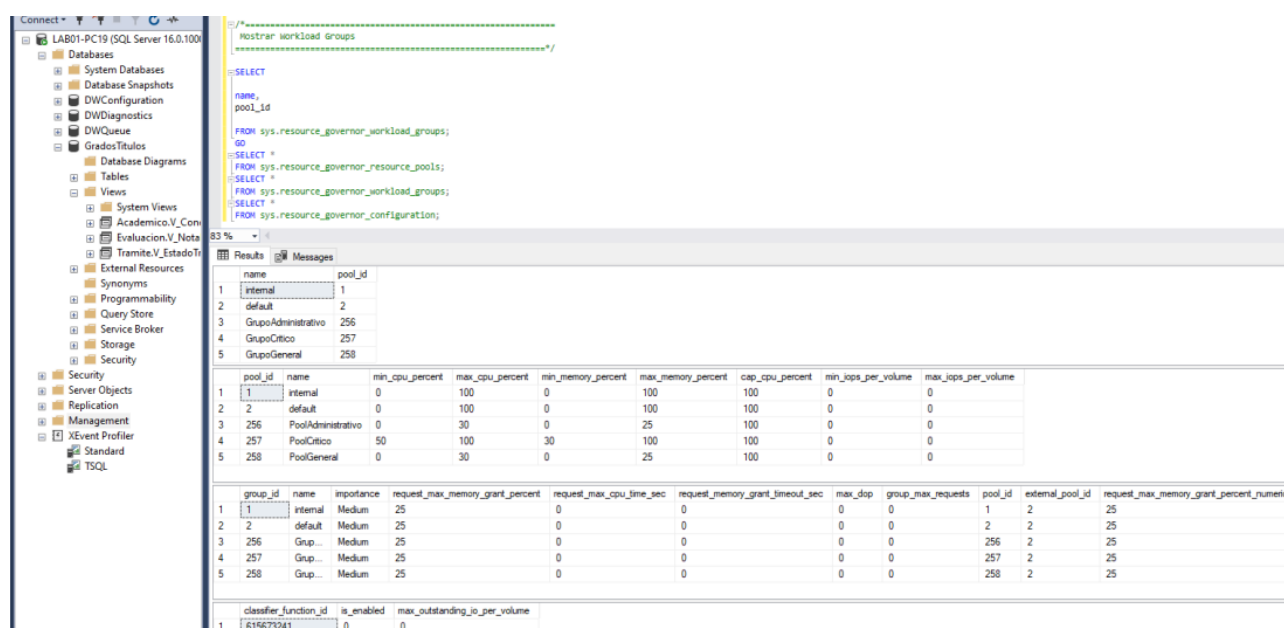
Proyecto 2: Priorización de procesos críticos

1. Descripción del requerimiento

Durante el cierre del semestre académico, los procesos relacionados con la emisión de grados y diplomas requieren mayor disponibilidad de recursos que las consultas ejecutadas por los usuarios generales. Para atender esta necesidad, se implementará una solución basada en **Resource Governor**, orientada a administrar la carga de trabajo del servidor mediante la creación de grupos con diferentes niveles de prioridad. La solución deberá contemplar:

- La creación de grupos de carga diferenciados según el tipo de proceso.
- La asignación de mayor prioridad a las operaciones críticas del sistema.
- La restricción del consumo de recursos para los procesos de menor prioridad.
- La evaluación del impacto de la configuración aplicada sobre el rendimiento general del servidor.

2. Script



```

--Mostrar Workload Groups
--=====*/

SELECT
    name,
    pool_id
FROM sys.resource_governor_workload_groups;
GO
--=====
SELECT *
FROM sys.resource_governor_resource_pools;
--=====
SELECT *
FROM sys.resource_governor_workload_groups;
--=====
SELECT *
FROM sys.resource_governor_configuration;
--=====

```

pool_id	name	min_cpu_percent	max_cpu_percent	min_memory_percent	max_memory_percent	cap_cpu_percent	min_iops_per_volume	max_iops_per_volume
1	internal	0	100	0	100	100	0	0
2	default	0	100	0	100	100	0	0
3	PoolAdministrativo	0	30	0	25	100	0	0
4	PoolCritico	50	100	30	100	100	0	0
5	PoolGeneral	0	30	0	25	100	0	0

group_id	name	importance	request_max_memory_grant_percent	request_max_cpu_time_sec	request_memory_grant_timeout_sec	max_dop	group_max_requests	pool_id	external_pool_id	request_max_memory_grant_percent_numeric
1	internal	Medium	25	0	0	0	0	1	2	25
2	default	Medium	25	0	0	0	0	2	2	25
3	Grupo...	Medium	25	0	0	0	0	256	2	25
4	Grupo...	Medium	25	0	0	0	0	257	2	25
5	Grupo...	Medium	25	0	0	0	0	258	2	25

classifier_function_id	is_enabled	max_outstanding_io_per_volume
1	0	0

3. Justificación técnica

Se implementó una estrategia de administración de recursos mediante **Resource Governor**, configurando dos **Resource Pools** independientes: uno orientado a los procesos críticos relacionados con la emisión de diplomas y otro destinado a las consultas generales realizadas por los usuarios. Además, se desarrolló una función clasificadora que identifica automáticamente las conexiones según la aplicación utilizada y las asigna al **Workload Group** correspondiente. Gracias a esta configuración, los procesos prioritarios disponen de una mayor asignación de CPU y memoria, mientras que las tareas de menor prioridad mantienen un consumo controlado, contribuyendo a un mejor rendimiento del servidor.

4. Buenas prácticas

- Configurar **Resource Pools** independientes para distribuir los recursos del servidor de acuerdo con la prioridad de cada carga de trabajo.
- Definir límites apropiados de CPU y memoria para cada grupo, evitando que los procesos secundarios afecten a las operaciones críticas.

- Implementar una función clasificadora que automatice la asignación de las conexiones a los grupos de carga correspondientes.
- Aplicar los cambios mediante **ALTER RESOURCE GOVERNOR RECONFIGURE** para que la configuración entre en funcionamiento correctamente.
- Comprobar el estado y la configuración de **Resource Governor** utilizando las vistas de administración y catálogo del sistema.
- Emplear nombres descriptivos para los **Resource Pools** y **Workload Groups**, facilitando su administración y mantenimiento a largo plazo.

Proyecto 3: Monitoreo del consumo de recursos

1. Descripción del requerimiento

La Oficina de Tecnologías de la Información de la **Universidad Peruana Los Andes** requiere supervisar el comportamiento de los recursos utilizados por cada grupo de trabajo configurado en **SQL Server**. Para ello, se desarrollará un conjunto de consultas que permita monitorear el consumo de CPU, memoria, número de solicitudes y sesiones activas mediante las **vistas de administración dinámica (DMV)** asociadas a **Resource Governor**. Finalmente, se generará un reporte consolidado que facilite el análisis del rendimiento del servidor y el seguimiento de la distribución de recursos entre los distintos grupos de carga.

2. Script

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the Object Explorer shows the server structure for 'LAB01-PC19 (SQL Server 16.0.1000.1)'. The central pane shows a SQL query in 'SQLQuery29.sql' that uses various DMV queries to monitor Resource Governor settings and usage. The bottom pane shows the results of the query, which are divided into two sections: the first shows the status of the classifier function, and the second shows a detailed view of the resource pools and workload groups.

SQL Query:

```

SELECT
    GETDATE() AS FechaMonitoreo,
    COUNT(*) AS TotalSesiones,
    SUM(cpu_time) AS CpuTotal,
    SUM(logical_reads) AS LecturasLogicas,
    SUM(reads) AS LecturasFisicas,
    SUM(writes) AS Escrituras
FROM sys.dm_exec_requests;
GO

```

Results:

classifier_function_id	is_enabled
615673241	0

pool_id	name	min_cpu_percent	max_cpu_percent	min_memory_percent	max_memory_percent
1	internal	0	100	0	100
2	default	0	100	0	100
3	256	0	30	0	25
4	257	50	100	30	100

group_id	name	pool_id
1	internal	1
2	default	2
3	GrupoAdministrativo	256
4	GrupoCritico	257

name	total_request_count	active_request_count	queued_request_count	total_cpu_limit_violation_count
1 internal	0	0	0	0

session_id	status	cpu_time	total_elapsed_time	logical_reads	reads	writes	command
1 23	background	3187	17113610	0	0	0	LAZY WRITER
2 20	background	453	17113613	0	0	0	RECOVERY WRITER
3 25	background	390	17113609	0	0	0	LOG WRITER
4 24	background	375	17113609	0	0	0	LOG WRITER

session_id	login_name	host_name	program_name	status
13 63	NT AUT...	LAB01-PC19	LAB01-PC19\mpdwsvc (476) - (E5D199...	sleeping
14 64	NT AUT...	LAB01-PC19	LAB01-PC19\mpdwsvc (476) - (E5D199...	sleeping

physical_memory_in_use_kb	locked_page_allocations_kb	virtual_address_space_reserved_kb	virtual_address_space_committed_kb
1 879192	0	58087616	1073084

FechaMonitoreo	TotalSesiones	CpuTotal	LecturasLogicas	LecturasFisicas	Escrituras
1 2026-07-08 15:06:00.237	60	4869	7929	12	59

group_id	name	importance	request_max_memory_grant_percent	request_max_cpu_time_sec	request_memory_grant_timeout_sec	max_dop	group_max_requests	pool_id	external_pool_id	request_max_memory_grant_percent_numeric
1 1	internal	Medium	25	0	0	0	0	1	2	25
2 2	default	Medium	25	0	0	0	0	2	2	25
3 256	GrupoAdministrativo	Medium	25	0	0	0	0	256	2	25
4 257	GrupoCritico	Medium	25	0	0	0	0	257	2	25
5 258	GrupoGeneral	Medium	25	0	0	0	0	258	2	25

session_id	request_id	start_time	status	command	sql_handle	statement_start_offset	statement_end_offset	plan_handle	database_id	user_id	connection_id	blocking_session_id	wait_type	wait_time	last_wait_type	wait_resource	open_transaction_count	open_result_set_count
1 1	0	2026-07-08 15:04:22.967	sleeping	TASK MANAGER	NULL	NULL	NULL	NULL	1	1	NULL	0	NULL	0	MISCELLANEOUS	0	0	1
2 2	0	2026-07-08 15:04:22.967	sleeping	TASK MANAGER	NULL	NULL	NULL	NULL	1	1	NULL	0	NULL	0	MISCELLANEOUS	0	0	1
3 3	0	2026-07-08 15:04:22.967	sleeping	TASK MANAGER	NULL	NULL	NULL	NULL	1	1	NULL	0	NULL	0	MISCELLANEOUS	0	0	1
4 4	0	2026-07-08 15:04:22.967	sleeping	TASK MANAGER	NULL	NULL	NULL	NULL	1	1	NULL	0	NULL	0	MISCELLANEOUS	0	0	1
5 5	0	2026-07-08 15:04:22.967	sleeping	TASK MANAGER	NULL	NULL	NULL	NULL	1	1	NULL	0	NULL	0	MISCELLANEOUS	0	0	1
6 6	0	2026-07-08 15:04:22.967	sleeping	TASK MANAGER	NULL	NULL	NULL	NULL	1	1	NULL	0	NULL	0	MISCELLANEOUS	0	0	1
7 7	0	2026-07-08 15:04:27.980	sleeping	TASK MANAGER	NULL	NULL	NULL	NULL	1	1	NULL	0	NULL	0	MISCELLANEOUS	0	0	1
8 8	0	2026-07-08 14:59:27.497	sleeping	TASK MANAGER	NULL	NULL	NULL	NULL	1	1	NULL	0	NULL	0	MISCELLANEOUS	0	0	1
9 9	0	2026-07-08 15:04:27.980	sleeping	TASK MANAGER	NULL	NULL	NULL	NULL	1	1	NULL	0	NULL	0	MISCELLANEOUS	0	0	1
10 10	0	2026-07-08 15:04:27.980	sleeping	TASK MANAGER	NULL	NULL	NULL	NULL	1	1	NULL	0	NULL	0	MISCELLANEOUS	0	0	1

3. Justificación técnica

Se implementó un conjunto de consultas basado en las **Dynamic Management Views (DMV)** de **SQL Server** con el propósito de supervisar el comportamiento de los grupos de trabajo y el consumo de recursos del servidor. A través de estas consultas es posible obtener información sobre el uso de CPU, memoria, solicitudes en ejecución, sesiones activas y diversas métricas del motor de base de datos. La información recopilada permite evaluar el rendimiento del sistema, identificar posibles cuellos de botella y apoyar la toma de decisiones para optimizar los procesos institucionales.

4. Buenas prácticas

- Utilizar las **DMV** oficiales de **SQL Server** para obtener información confiable y actualizada sobre el estado del servidor.
- Organizar las consultas de monitoreo por categorías, como CPU, memoria, sesiones y solicitudes, facilitando el análisis de los resultados.
- Elaborar un reporte consolidado que reúna los principales indicadores de rendimiento para simplificar su revisión.
- Emplear nombres descriptivos y comentarios dentro de los scripts para mejorar su comprensión, mantenimiento y reutilización.
- Realizar un monitoreo periódico del consumo de CPU y memoria con el fin de detectar oportunamente problemas de rendimiento y aplicar medidas de optimización cuando sea necesario.

